

**PONTIFICIA UNIVERSIDAD
CATÓLICA DEL PERÚ**

FACULTAD DE CIENCIAS SOCIALES



Impacto de la disponibilidad de ChatGPT en el desarrollo de software orientado a
Ciencia de Datos entre 2020 y 2023

Tesis para obtener el título profesional de Licenciado en Economía presentado por:

Nicho Rosado, Jesus Alberto

Janampa Aparicio, Karl
Willem

Asesor(es):

Quispe Rojas, Alexander Wilder

Lima, 2026

Agradecimientos

La culminación de este trabajo representa para mí un profundo sentimiento de satisfacción y orgullo. Este logro marca un antes y un después en mi trayectoria académica, constituyéndose en un hito que atesoraré siempre. Deseo expresar mi más sincero agradecimiento a mi madre Magali, mi padre Jesús, mi hermano Mathías y mi abuela Luz, cuyo apoyo incondicional, confianza y constante motivación fueron fundamentales para perseverar frente a las adversidades. Asimismo, guardo un especial recuerdo de todas las personas que formaron parte de este camino en la PUCP (profesores, compañeros y personal administrativo), quienes contribuyeron a que esta etapa estuviera llena de aprendizajes, desafíos y valiosas oportunidades. Finalmente, agradezco de manera especial a mi gran amigo Karl y a nuestro asesor, el profesor Alexander Quispe, con quienes conformé un excelente equipo de trabajo para la elaboración de este producto final.

Jesus Nicho

Agradezco a todas las personas que hicieron posible la culminación de esta tesis, la cual representa un hito en mi crecimiento personal y profesional, al constituir una etapa transformadora que fortaleció mi compromiso, el aprendizaje en el área, la experiencia en investigación, el trabajo colaborativo y mi perspectiva sobre los inicios de la vida laboral y creativa. Valoro el respaldo incondicional de mis padres, Selene y Kleber, cuya confianza, formación en valores y perseverancia han sido fundamentales en mi desarrollo, así como la motivación constante de mis hermanos, quienes confiaron en mí incluso en momentos de duda. Reconozco también la guía de mi asesor, Alexander, por su orientación académica, compromiso y motivación, además del aporte de mi compañero Jesús, quien enriqueció mi conocimiento y amplió mi visión hacia la innovación. Finalmente, destaco el apoyo firme y constante de Camila durante este proceso. Concluir esta etapa cierra un ciclo significativo y refleja una versión más sólida de mí mismo, constituyendo un motivo de orgullo personal.

Karl Janampa

Resumen

En los últimos años, la Inteligencia Artificial ha comenzado a transformar de manera significativa la producción de software y la forma en que los desarrolladores interactúan con herramientas de programación. En particular, la aparición de ChatGPT ha proporcionado a los programadores un nuevo recurso para el análisis, la generación y la depuración de código, así como para la resolución de problemas técnicos y la aceleración de procesos de aprendizaje. Este tipo de herramientas, basadas en modelos de lenguaje de gran escala, permite sugerir fragmentos de código, explicar funciones complejas y asistir en la documentación de proyectos, lo cual puede reducir los tiempos de desarrollo y mejorar la productividad de los equipos de programación. En este contexto, el objetivo del presente estudio es evaluar el impacto de ChatGPT en la producción de software durante el periodo comprendido entre el primer trimestre de 2020 y el cuarto trimestre de 2023. Para ello, se utiliza la base de datos GitHub Innovation Graph, que proporciona información agregada sobre actividad de desarrollo a nivel internacional. La estrategia empírica se basa en la aplicación de los métodos de Diferencias en Diferencias, Control Sintético y Diferencias en Diferencias Sintética con el fin de identificar cambios en la actividad de desarrollo asociados con la disponibilidad del servicio. Los resultados muestran un impacto diferenciado en el número de *unique pushers* por cada 100 mil habitantes en los países con acceso a ChatGPT en comparación con aquellos donde el servicio no está disponible.

Palabras clave: chatgpt, LLM, diferencia en diferencias sintética

Abstract

In recent years, Artificial Intelligence has begun to significantly transform software production and the way developers interact with programming tools. In particular, the emergence of ChatGPT has provided programmers with a new resource for code analysis, generation, and debugging, as well as for solving technical problems and accelerating learning processes. Tools based on large language models can suggest code snippets, explain complex functions, assist in the documentation of projects, and support developers during different stages of the software development cycle. These capabilities may reduce development time, improve productivity, and facilitate knowledge acquisition among programmers with different levels of experience. Within this context, the objective of this study is to evaluate the impact of ChatGPT on software production during the period from the first quarter of 2020 to the fourth quarter of 2023. To accomplish this, the study uses the GitHub Innovation Graph database, which provides aggregated information on software development activity across countries. The empirical strategy relies on the application of Difference-in-Differences, Synthetic Control, and Synthetic Difference-in-Differences methods in order to identify changes in development activity associated with the availability of the service. The results show a differentiated impact on the number of *unique pushers* per 100,000 inhabitants in countries that have access to ChatGPT compared with those where the service is not available.

Keywords: chatgpt, LLM, synthetic difference in differences

Índice de contenidos

Agradecimientos	2
Resumen	3
1. Introducción	8
1.1. Planteamiento del problema	8
1.2. Motivación/Justificación	9
1.3. Hipótesis.....	10
2. Revisión de Literatura	10
2.1. ChatGPT: Un Modelo de Lenguaje de Gran escala	10
2.2. Las versiones de ChatGPT.....	10
2.3. Capacidades de ChatGPT.....	11
2.4. Evidencia empírica	12
3. Datos.....	15
4. Estadísticos descriptivos.....	16
5. Metodología	18
5.1. Diferencia en Diferencias (DiD)	18
5.2. Control Sintético (CS)	18
5.3. Diferencia en Diferencias Sintética (SDiD)	19
5.4. Inferencia.....	20
6. Resultados	21
7. Conclusiones y recomendaciones de política	23
8. Tablas	25
9. Figuras	27

Índice de Figuras

Figura 1 Distribución de lenguajes de programación más populares en países con ChatGPT disponible (2020-2023)	27
Figura 2 Distribución de lenguajes de programación más populares en países sin ChatGPT disponible (2020-2023)	27
Figura 3 Distribución de unique pushers según lenguaje en países con ChatGPT (2020–2023)	28
Figura 4 Distribución de unique pushers según lenguaje en países sin ChatGPT (2020–2023)	29
Figura 5 Tendencias y ponderaciones estimadas de TypeScript unique pushers por cada 100 mil personas.....	30
Figura 6 Tendencias y ponderaciones estimadas de JavaScript unique pushers por cada 100 mil personas.....	31
Figura 7 Tendencias y ponderaciones estimadas de Python unique pushers por cada 100 mil personas	32
Figura 8 Tendencias y ponderaciones estimadas de Java unique pushers por cada 100 mil personas	33
Figura 9 Tendencias y ponderaciones estimadas de Go unique pushers por cada 100 mil personas	34
Figura 10 Tendencias y ponderaciones estimadas de Ruby unique pushers por cada 100 mil personas	35
Figura 11 Tendencias y ponderaciones estimadas de PHP unique pushers por cada 100 mil personas	36
Figura 12 Tendencias y ponderaciones estimadas de C# unique pushers por cada 100 mil personas	37
Figura 13 Tendencias y ponderaciones estimadas de C++ unique pushers por cada 100 mil personas	38
Figura 14 Tendencias y ponderaciones estimadas de C unique pushers por cada 100 mil personas	39

Índice de Tablas

Tabla 1 <i>Estadísticos descriptivos del número de unique pushers por lenguaje de programación en países del grupo de control (periodos pre y postratamiento)</i>	25
Tabla 2 <i>Estadísticos descriptivos del número de unique pushers por lenguaje de programación en países del grupo de tratamiento (periodos pre y postratamiento)</i> ..	25
Tabla 3 <i>Impacto heterogéneo de ChatGPT en el desarrollo de software vinculado a Ciencia de Datos</i>	26

1. Introducción

1.1. Planteamiento del problema

Antes de la introducción de ChatGPT, la industria del software ya estaba comprometida con la búsqueda de herramientas y metodologías más eficientes y automatizadas, puesto que pretendía reducir la carga cognitiva de los desarrolladores y acelerar los ciclos de desarrollo. Estos esfuerzos se han reflejado en el incremento de ocho veces más de las patentes de Inteligencia Artificial comparadas con las demás (World Intellectual Property Organization, 2022). Respecto a esta cifra, resaltan los avances en el subcampo de Procesamiento y Generación de Lenguaje, el cual se enfoca en desarrollar modelos de aprendizaje automático entrenados con lenguaje humano para analizar, predecir y generar texto.

La arquitectura *transformer* se ha vuelto fundamental en dicho subcampo, debido a que optimiza tanto el entrenamiento como la implementación de modelos centrados en la interacción entre palabras en el lenguaje natural. OpenAI ha liderado la construcción de esta arquitectura con el desarrollo de la serie de modelos GPT (Generative Pretrained Transformer), ejemplos extraordinarios de Grandes Modelos de Lingüísticos (LLM). Dichos modelos, diseñados para procesar y generar texto coherente y relevante, han sido implementados en ChatGPT, un chatbot que aprovecha su habilidad para generar respuestas basadas en una amplia base de datos de temas variados, incluyendo consultas de programación (OpenAI, 2018). Lo cual ha impulsado su uso en la industria del software al cumplir el rol de orientador y asistente sobre diferentes lenguajes de programación.

Desafortunadamente, se ha identificado a un grupo que no posee la disponibilidad del servicio del chatbot. La razón principal de ello se debe al establecimiento de restricciones de acceso a ChatGPT en países como China, Irán, Siria, Rusia, Corea del Norte, Cuba e Italia, los cuales han sido promovidos por preocupaciones de seguridad nacional, privacidad o cumplimiento de regulaciones locales (Martínez, 2023). Al respecto, algunos autores han señalado que esta limitación ha inhibido la innovación y provocado que estas regiones se mantengan atrasadas en la economía digital global, dado que los desarrolladores de software no han podido aprovechar los beneficios del chatbot (Del Río-Chanona et al., 2023).

Ante ello, recientes estudios han manifestado los desafíos de ChatGPT. Gallea (2023) observa un rendimiento deficiente del chatbot en responder a consultas complejas de código en los lenguajes de programación Python y R. Brynjolfsson et al. (2023) mencionan que las alucinaciones son responsables de que herramientas populares basadas en modelos de lenguaje, como ChatGPT, generen información falsa o engañosa de forma impredecible. Demirci et. al (2024) ponen de relieve el problema de la reducción de oferta de trabajos ligados a la codificación para trabajadores independientes y enfatizan la necesidad de encontrar soluciones. Del Río-Chanona et al. (2023) advierten que, al proporcionar información de manera eficiente y privada, ChatGPT puede reducir significativamente la disponibilidad de datos públicos, lo que plantea un desafío para la obtención de datos de entrenamiento para futuros modelos y pone en cuestión la sostenibilidad a largo plazo de estas tecnologías avanzadas.

A pesar de la popularidad y la rápida adopción de herramientas de IA como ChatGPT en el desarrollo de software, existe una notable falta de investigaciones empíricas que evalúen de manera sistemática su impacto real. Este estudio se propone llenar ese vacío y busca determinar si ChatGPT ha contribuido a acelerar la

generación de software en los últimos años. Además, se examinarán tanto los beneficios como los desafíos asociados al uso de la generación automatizada de código. Con este enfoque, se pretende proporcionar una evaluación integral que resalte los beneficios y también los riesgos potenciales, ofreciendo orientación clara para futuras políticas y prácticas sobre la integración de IA en la industria.

La variable de interés del estudio es el número de *unique pushers* por cada 100 mil habitantes en GitHub, el cual representa la frecuencia de desarrolladores únicos que, como mínimo, subieron un código en la plataforma en cierto trimestre. Esta variable será utilizada como un *proxy* de la actividad de desarrollo de software. Así, el uso de esta métrica permitirá evaluar el impacto heterogéneo de la disponibilidad de ChatGPT en los países con accesibilidad a este servicio en comparación con aquellos que no lo disponen. Analizando los *unique pushers*, se puede inferir cómo ChatGPT influye en la productividad y la frecuencia de contribuciones de los desarrolladores de software en los diez lenguajes de programación más utilizados en ciencia de datos.

1.2. Motivación/Justificación

La integración de la inteligencia artificial en el desarrollo de software, particularmente a través de herramientas avanzadas como ChatGPT, está redefiniendo las metodologías tradicionales de este campo. Con un mercado global de software que se proyecta superar los 600 mil millones de dólares para 2025, la eficiencia y la innovación en el desarrollo de software no solo son deseables, sino imperativas para mantener la competitividad global (Statista, 2022). La capacidad de ChatGPT para generar código y asistir en la solución de problemas de programación de manera eficiente presenta una oportunidad crucial para mejorar estos procesos. Sin embargo, la adopción y las implicaciones de esta tecnología no se han evaluado completamente, lo que destaca la necesidad crítica de una investigación exhaustiva.

Este estudio abordará la comprensión de cómo las herramientas basadas en modelos de lenguaje, específicamente, ChatGPT pueden afectar la productividad y la calidad en el desarrollo de software. A pesar del uso creciente de IA en muchos sectores, pocos estudios han medido sistemáticamente el impacto real de estas tecnologías en el desarrollo de software, reconociendo su falta de eficacia operativa y precisión del código generado para ciertos programas. Este estudio pretende llenar ese vacío proporcionando un análisis empírico y detallado sobre los beneficios y los desafíos de integrar ChatGPT en los flujos de trabajo de desarrollo.

Además de evaluar el impacto práctico, este estudio también enmarcará las implicaciones de la implementación de IA en el desarrollo de software. Con un aumento del 14% en la adopción de IA en las empresas a nivel global desde 2021 (Chui, 2022), es crucial entender cómo la dependencia de estas tecnologías afecta la dinámica laboral, la seguridad de los datos y la equidad en el acceso a herramientas tecnológicas avanzadas. Asimismo, los problemas de sesgo en los modelos de IA y la calidad del código que estos generan son preocupaciones que deben ser señaladas para evitar repercusiones negativas a largo plazo.

La restricción del acceso a tecnologías como ChatGPT en varios países debido a preocupaciones de seguridad nacional y privacidad subraya la necesidad de estudiar más a fondo las políticas y prácticas que podrían fomentar o impedir la adopción equitativa de IA en el desarrollo de software (ONU, 2023). Este estudio no solo aportará a la literatura académica, proporcionando un análisis crítico de las ventajas y limitaciones de ChatGPT, sino que también ofrecerá recomendaciones para políticas

y estrategias que promuevan un uso más justo y eficiente de la IA en el desarrollo de software a nivel global.

1.3. Hipótesis

La disponibilidad de ChatGPT en un país influiría positivamente en el desarrollo de software en los diez lenguajes de programación más utilizados en ciencia de datos. Así, los países con acceso a ChatGPT experimentarían un aumento en el número de *unique pushers* por cada 100 mil habitantes (es decir, desarrolladores que realizaron al menos un aporte de código en GitHub) en comparación con aquellos sin acceso al servicio. Asimismo, este efecto sería más notable en lenguajes como Python y JavaScript, debido a que el modelo ha sido entrenado con una gran cantidad de datos provenientes de estos lenguajes, incluyendo ejemplos de código, documentación técnica y foros especializados. Esta exposición durante el entrenamiento, sumada a su sintaxis accesible y la abundante disponibilidad de recursos, permite que ChatGPT ofrezca respuestas más precisas y útiles sobre la resolución de problemas de codificación en tiempo real.

2. Revisión de Literatura

Los Modelos de Lenguaje Grande, cuya sigla en inglés es LLM (Large Language Models), son sistemas basados en la inteligencia artificial, capaces de generar y comprender textos en lenguaje natural o humano con un alto grado de precisión. Estos sistemas son entrenados con masivas cantidades de datos textuales con el propósito de aprender y mejorar patrones en diferentes actividades a un nivel profundo. Los LLM están vinculados con el Machine Learning, debido a que utilizan algoritmos avanzados, especialmente redes neuronales profundas, para el procesamiento y análisis de datos. Estos algoritmos son capaces de aprender y adaptarse, mejorando su rendimiento con cada entrenamiento y tiempo.

2.1. ChatGPT: Un Modelo de Lenguaje de Gran escala

Según Tlili et al. (2023), ChatGPT es un chatbot con interfaz de inteligencia artificial conversacional. Este modelo de lenguaje fue desarrollado por OpenAI con base en una arquitectura GPT (Generative Pre-trained Transformer). Es considerado un LLM, como mencionan Kalla et. al. (2023) y Abu et. al (2023) por su estructura de redes neuronales profundas con múltiples capas de transformadores y que están diseñados para procesar y generar respuestas coherentes en un lenguaje natural. Según Shen et. al (2024), sus algoritmos han sido entrenados con extensas bases de datos textuales obtenidas de internet, demostrando una capacidad impresionante para responder preguntas y realizar diversas tareas con calidad competitiva a la humana.

2.2. Las versiones de ChatGPT

Cada versión de ChatGPT, según OpenAI (2024), ha mantenido la estructura de aprendizaje de los LLM, lo que ha permitido una evolución en cuanto a la precisión, reconocimiento y velocidad de respuesta. Asimismo, las últimas versiones han empezado a incluir nuevas herramientas versátiles para abarcar mayores áreas y

también aumentar la ciberseguridad de cada modelo. A continuación, se destacan las tres versiones más recientes y relevantes siguientes¹:

- **Versión GPT-3.5:** Este modelo comprende, genera lenguaje natural y código de manera avanzada. Se caracteriza por su optimización para el chat mediante APIs Chat Completions, aunque también son eficientes para otras tareas que no involucran interacción escrita. Desde el lanzamiento de esta versión, en noviembre de 2022 por OpenAI, ha demostrado ser una herramienta eficaz debido a su potencial, capacidad e impacto en el desarrollo de software. Por ello, gracias a su mayor cobertura y acceso gratuito, la mayoría de los estudios disponibles sobre el impacto de ChatGPT en el desarrollo de software se han centrado en el modelo GPT 3.5.
- **Versión GPT-4.0:** Este modelo multimodal avanzado, genera texto a partir de entradas multimedia y puede resolver problemas compuestos con mayor eficiencia que las versiones anteriores, gracias a su conocimiento general más amplio y habilidades de razonamiento avanzado. También, se caracteriza por mantener la optimización en la interacción por chat con el usuario y la implementación del Code Interpreter, mejorando significativamente la comprensión y ofreciendo respuestas más precisas y detalladas. Fue lanzado en marzo de 2023 con acceso limitado mediante una suscripción de paga.
- **Versión GPT-4o:** Conocido también como *Omni* por la letra o, este modelo multimodal mantiene la inteligencia de la versión anterior y la capacidad de generar respuestas a partir de diversos archivos multimedia. Se diferencia por ser mucho más eficiente, generando textos 2 veces más rápido que su predecesor y con un costo de acceso menor. Se caracteriza por su mejor visión y rendimiento en traducción de múltiples idiomas. Fue lanzado en mayo de 2024 generando un gran impacto en la comunidad de desarrolladores².

2.3. Capacidades de ChatGPT

Diversos autores han propuesto métricas y benchmarks con el objetivo de estimar la precisión y calidad del código generado por ChatGPT. En esta línea, Chai et al. (2024) presentan MCEVAL, un benchmark multitarea y multilingüe para la evaluación de generación de código, que abarca 40 lenguajes de programación y más de 16.000 ejemplos diseñados manualmente. A diferencia de benchmarks anteriores como HumanEval o MultiPL-E, los cuales se centran principalmente en Python o dependen de traducciones automáticas a otros lenguajes, MCEVAL fue construido desde cero mediante anotaciones humanas. Dicho benchmark incluye tareas específicas de generación, explicación y verificación de código, lo que garantiza una evaluación más representativa de las capacidades multilingües de los modelos de lenguaje de gran escala (LLM). En cuanto al desempeño de ChatGPT-3.5 Turbo, los resultados evidencian una alta variabilidad según el lenguaje de programación: se observan tasas de éxito relativamente elevadas en lenguajes populares como Python (60%), Java (71.7%), PHP (68%) y Ruby (76%), mientras que se identifican limitaciones significativas en lenguajes menos difundidos como Elisp (26%), Scheme (46%),

¹ Disponibles hasta la fecha de realizado el estudio

² No será considerado en el análisis, puesto que los datos disponibles son limitados hasta el año 2023.

Markdown (24%) y VimL (24%). Estos hallazgos sugieren que, si bien el modelo alcanza un desempeño sólido en lenguajes de propósito general ampliamente utilizados, aún enfrenta desafíos importantes en lenguajes con menor presencia en los datos de entrenamiento.

De manera consistente, Li y Krishnamachari (2024) muestran que el rendimiento de ChatGPT-3.5 Turbo depende fuertemente del lenguaje de programación empleado. Para ello, utilizan un benchmark compuesto por 1.475 problemas extraídos de la plataforma LeetCode, ampliamente utilizado para la preparación de entrevistas técnicas, y evalúan la capacidad del modelo para generar soluciones correctas en seis lenguajes de programación: Python (lenguaje base), Java, C++, Elixir, Erlang y Racket. En un subconjunto de problemas previamente resueltos en Python, el modelo logró resolver el 70% de los casos en Java y el 50% en C++, e incluso presentó soluciones exitosas en estos lenguajes tras haber fallado en Python, lo que sugiere una sensibilidad del modelo a las características específicas de cada lenguaje. En contraste, no resolvió ningún problema en lenguajes funcionales menos comunes como Elixir, Erlang y Racket, resultado que los autores atribuyen a su baja representación en los datos de entrenamiento y las diferencias en sus paradigmas y estructuras sintácticas. En conjunto, estos resultados confirman que el desempeño de ChatGPT-3.5 Turbo es altamente dependiente del lenguaje de programación considerado.

Por su parte, Buscemi (2023) propone un benchmark *ad hoc* para evaluar la capacidad de generación de código de ChatGPT-3.5 Turbo en 10 lenguajes de programación, a partir de 40 tareas organizadas en cuatro categorías: algoritmos simples, juegos, seguridad informática y ciencia de datos. El benchmark se construyó mediante consultas diseñadas por el propio autor y ejecutadas a través de la API de OpenAI, solicitando explícitamente la implementación del código sin el uso de librerías externas y la inclusión de una única prueba por tarea. A diferencia de benchmarks existentes como HumanEval, MBPP o CodeContests, que suelen limitarse a pocos lenguajes o presentan una cobertura de pruebas reducida, este enfoque permite una comparación multilenguaje más exhaustiva, con 10 ejecuciones por tarea y lenguaje, lo que totaliza 4,000 evaluaciones individuales. En la categoría de ciencia de datos, Julia se posicionó como el lenguaje con mayor tasa de éxito en la generación de código ejecutable, alcanzando un 81.5%, seguido por R y Python. Estos resultados sugieren que ChatGPT-3.5 Turbo presenta un mejor desempeño funcional en lenguajes de alto nivel diseñados específicamente para análisis de datos y modelado estadístico, lo que refleja una relación entre la familiaridad del modelo con dichos lenguajes, posiblemente asociada a su popularidad en los datos de entrenamiento, y su capacidad para resolver tareas complejas de ciencia de datos de manera.

2.4. Evidencia empírica

La rápida adopción de la inteligencia artificial (IA) en diversos campos ha generado un considerable interés académico, centrado en evaluar sus impactos concretos en la economía global, la transformación de las dinámicas laborales y la aceleración de la innovación tecnológica. Esta subsección examina estudios empíricos que investigan las repercusiones directas de tecnologías disruptivas, particularmente los modelos generativos de lenguaje como ChatGPT. La literatura seleccionada utiliza metodologías de evaluación de impacto que permiten medir con precisión el efecto de estas tecnologías en variables clave como la productividad, la calidad del trabajo y los cambios en los mercados laborales. Los estudios revisados identifican tanto impactos

positivos como negativos, lo que hace relevante considerar el impacto neto de estas tecnologías.

Brynjolfsson et al. (2023) exploran la implementación de una herramienta de IA generativa como asistente conversacional en servicios de atención al cliente y su impacto en la productividad. Utilizando un modelo de regresión de diferencia en diferencias (DiD), el análisis se basa en datos de 5,179 agentes de soporte al cliente de una empresa Fortune 500. La variable dependiente es la productividad de los agentes, medida por el número de problemas resueltos por hora, y las variables independientes incluyen el acceso a la herramienta de IA y características del agente como experiencia y habilidad. Los hallazgos revelan un incremento del 14% en la productividad, especialmente notable en trabajadores menos experimentados y de menor habilidad, mejoras en el sentimiento del cliente y una reducción en las intervenciones gerenciales. Los resultados sugieren que la IA puede facilitar la diseminación de conocimiento tácito y acelerar el aprendizaje en nuevos empleados, aunque no se exploran los efectos globales sobre el empleo o los salarios.

Gallea (2023) investiga el impacto de la IA generativa en la dinámica laboral dentro de las comunidades de codificación en línea, utilizando como caso de estudio a ChatGPT y su efecto en Stack Overflow. El estudio emplea un modelo de diferencias en diferencias (DiD) para comparar las preguntas relacionadas con los lenguajes de programación Python y R antes y después del lanzamiento de ChatGPT, utilizando datos de Stack Overflow desde octubre de 2022 hasta marzo de 2023. La unidad de análisis se define por el lenguaje de programación y la semana, evidenciando una disminución significativa en el número semanal de preguntas publicadas, particularmente en el caso de Python, que actúa como grupo tratado debido al mayor desempeño de ChatGPT en este lenguaje, mientras que R funciona como grupo de control. Esta reducción sugiere que la disponibilidad de la IA estaría resolviendo de forma eficiente consultas rutinarias fuera de la plataforma. Además, se observa un aumento en la complejidad y la calidad de las preguntas restantes, lo que indica un desplazamiento hacia tareas que requieren mayor especialización y esfuerzo cognitivo. El estudio concluye que, si bien ChatGPT potencia la productividad al encargarse de tareas repetitivas, también plantea interrogantes sobre los efectos a largo plazo de una mayor dependencia de la IA en las habilidades de resolución de problemas de los programadores.

Liu et al. (2023) analizan el impacto del lanzamiento de ChatGPT como un choque exógeno en los mercados laborales online. Utilizan un modelo de DiD para analizar el volumen de transacciones de freelancers y trabajos en una plataforma de labor online. Las variables independientes incluyen la exposición a ChatGPT, el volumen de transacciones anterior y la calidad del servicio basada en calificaciones previas. La unidad de análisis comprende trabajos y freelancers clasificados por su nivel de exposición a ChatGPT. La base de datos proviene de una plataforma prominente de freelancers que facilita una variedad de servicios digitales y físicos. Las conclusiones señalan que, aunque el lanzamiento de ChatGPT resultó en una disminución significativa en el volumen de transacciones para trabajos expuestos, los freelancers que adaptaron sus servicios para complementar la tecnología IA experimentaron beneficios notables, resaltando la dualidad del impacto de la IA en el trabajo y la necesidad de más investigaciones para comprender plenamente sus efectos.

Wu et al. (2024) investigan el impacto del lanzamiento de ChatGPT en el retorno anormal acumulado (CAR) de las empresas cotizadas en China. Utilizando un modelo de DiD combinado con un estudio de eventos, la investigación evalúa cómo los lanzamientos de GPT-3.5 y GPT-4 influyeron positivamente en las firmas enfocadas

en ChatGPT. La unidad de análisis identifica a las compañías del índice A-share del mercado chino, con la variable dependiente siendo el CAR y las independientes incluyendo un indicador de tratamiento basado en la mención de ChatGPT en informes anuales y variables de tiempo post-lanzamiento. Los datos provienen de la base CSMAR y de índices de atención en Baidu, WeChat y Douyin. Las conclusiones indican que los eventos de lanzamiento de ChatGPT aumentaron significativamente los CAR de las empresas enfocadas en esta tecnología, especialmente bajo alta atención social, sugiriendo que la integración de IA generativa podría ser beneficiosa para las empresas y la formulación de políticas.

Del Rio-Chanona et al. (2023) analizan el impacto de los modelos de lenguaje como ChatGPT en la generación de datos públicos³. Utilizando un modelo de DiD, los investigadores examinaron las publicaciones en Stack Overflow en comparación con foros similares menos susceptibles de ser sustituidos por ChatGPT. La unidad de análisis es a nivel de plataforma y por semana, considerando las variaciones semanales en la actividad de publicación. Las variables independientes incluyen el tiempo transcurrido desde el lanzamiento de ChatGPT y los lenguajes de programación más populares. La base de datos comprende millones de publicaciones desde la creación de Stack Overflow hasta junio de 2023. Los hallazgos muestran una reducción del 16% en la actividad de publicación tras la introducción de ChatGPT, sugiriendo una significativa sustitución por parte de los usuarios que ahora prefieren utilizar modelos de lenguaje para resolver consultas. Esto plantea preocupaciones sobre la disminución de datos públicos disponibles para entrenar futuros modelos, destacando la necesidad de políticas que gestionen el impacto de los LLM en los bienes públicos digitales.

Demirci et al. (2024) examinan el impacto de la IA generativa (GenAI) en las plataformas de freelancing online, centrándose en cómo tecnologías como ChatGPT y otras herramientas de generación de imágenes han cambiado la demanda de trabajos susceptibles a la automatización. Utilizan un modelo de análisis de DiD, incluyendo versiones sintéticas y robustas doblemente, para evaluar estos efectos comparando los períodos antes y después de la introducción de GenAI. La unidad de análisis comprende las ofertas individuales de trabajo en la plataforma, clasificadas semanalmente por país y tipo de trabajo. La variable dependiente es el logaritmo del número de ofertas de trabajo, mientras que las variables independientes incluyen indicadores del período post-introducción de GenAI y un índice de volumen de búsqueda de Google (SVI) como proxy del interés público en estas tecnologías. La base de datos contiene más de 1.38 millones de ofertas de trabajo desde julio de 2021 hasta julio de 2023. Las conclusiones destacan una reducción significativa en la demanda de trabajos propensos a la automatización y generación de imágenes, sugiriendo que la GenAI está reemplazando funciones laborales específicas y subrayando la necesidad de estrategias para integrar la IA en los entornos laborales para mitigar posibles impactos negativos en el mercado laboral.

³ Los datos públicos (o bienes públicos digitales) se entienden como información y conocimiento generados por personas y puestos en la web de manera abierta, de modo que cualquiera puede acceder, usar, aprender de ellos y reutilizarlos sin excluir a otros ni reducir su valor. Algunos ejemplos representativos a plataformas son Wikipedia, Stack Overflow y Reddit, en las que las preguntas, respuestas y debates generados por los propios usuarios conforman un acervo colectivo de conocimiento, de carácter no rival y no excluyente, accesible a toda la sociedad y utilizado, además, como insumo para el entrenamiento de sistemas de inteligencia artificial.

Hui et al. (2023) examinan el impacto de la IA generativa, como ChatGPT, sobre el empleo y los ingresos de los freelancers en la plataforma en línea Upwork. Utilizando un diseño de DiD, este análisis empírico se centra en las ocupaciones más susceptibles a ser afectadas por la IA, como la redacción y edición. La unidad de análisis son los freelancers individuales, cuyos resultados se miden en términos del número de trabajos e ingresos mensuales. Las variables independientes incluyen un indicador post-introducción de ChatGPT, ocupaciones potencialmente afectadas y características de los freelancers como puntuaciones de *feedback* e ingresos mensuales pasados. La base de datos proviene de la API de Upwork, que ofrece acceso a perfiles completos de empleo y atributos relevantes. Las conclusiones refieren que la introducción de la IA tiene efectos negativos significativos en los freelancers en ocupaciones afectadas, disminuyendo tanto la cantidad de trabajos como los ingresos. Esto sugiere que la IA generativa puede actuar como un sustituto en el corto plazo, reduciendo la demanda general por trabajadores del conocimiento y posiblemente igualando las brechas de productividad entre trabajadores de diferentes calidades.

La literatura revisada indica que ChatGPT y otros modelos de lenguaje grandes han influido significativamente en diversas áreas, desde la generación de datos públicos hasta la dinámica laboral y la productividad. Los estudios destacan tanto beneficios como desafíos, subrayando la importancia de un análisis equilibrado del impacto neto. Sin embargo, la mayoría de estos estudios no se han centrado específicamente en el impacto de ChatGPT en la productividad del desarrollo de software en diferentes lenguajes de programación. Este estudio se propone llenar ese vacío, proporcionando un análisis empírico confiable que considera los beneficios y desafíos de integrar ChatGPT en los flujos de trabajo de desarrollo de software.

3. Datos

ChatGPT fue lanzado en noviembre de 2022 por el *startup* estadounidense OpenAI, cuya sede se encuentra en San Francisco. Desde su lanzamiento, la herramienta ha alcanzado una adopción masiva, siendo utilizada por millones de personas a nivel global (Kreitmeir & Raschky, 2023). Sin embargo, su disponibilidad no ha sido homogénea entre países, ya que algunos gobiernos han decidido restringir o bloquear su acceso debido a preocupaciones vinculadas a la protección de datos personales, la potencial difusión de desinformación, la ausencia de mecanismos efectivos de regulación de contenidos en línea y conflictos geopolíticos con Estados Unidos (BBC News Mundo, 2023; Ijaz, 2023). En el Anexo 1, se presenta un análisis detallado del estado de accesibilidad de ChatGPT por país.

Github⁴ es la plataforma de almacenamiento de código más grande en el mundo, utilizada para almacenar y trabajar conjuntamente en proyectos de codificación a través de repositorios. Para este trabajo, la totalidad de los datos mostrados provienen del base de datos Innovation Graph de GitHub, específicamente de la tabla *languages*, la cual contiene a los *unique pushers* a nivel de lenguaje de programación y por trimestre. Esta métrica, denominada *unique pushers*, representa a la cantidad total de desarrolladores únicos que, al menos, realizaron un *git push* (subida de código) a un

⁴ GitHub registra automáticamente y en tiempo real todas las modificaciones realizadas en los repositorios, incluyendo fechas, autores, archivos y líneas de código afectadas. Este historial permite el seguimiento detallado de las iteraciones y acciones de los equipos de desarrollo. En particular, los repositorios públicos de GitHub representan la mayoría; por lo que ofrecen una medición directa y accesible de la actividad laboral de millones de desarrolladores a nivel mundial (Kreitmeir & Raschky, 2023).

repositorio a GitHub con un lenguaje de programación determinado. Los datos abarcan desde el primer trimestre de 2020 hasta el cuarto trimestre de 2024, pero se tomará solo hasta el año 2023, y están desglosados por lenguaje de programación, incluyendo un total de 50 lenguajes. De estos, se han mantenido los 10 más populares en el campo de la ciencia de datos.

En vista de la gran cantidad de lenguajes de programación recurridos en la ciencia de datos, la selección de los 10 más utilizados se ha basado en diversos rankings y en la cantidad de observaciones disponibles. Así, diferentes fuentes en este campo coinciden en que los lenguajes más importantes son JavaScript, Java, Python, TypeScript, C, C++, C#, PHP, Ruby y Go (Coursera Staff, 2024; Data Camp, 2023; Rice CS, 2023). Aunque otros lenguajes como SQL, R, Julia y Matlab también son reconocidos, fueron excluidos de esta selección debido a su baja cantidad de observaciones, lo que limita su representatividad y análisis comparativo.

Además, el filtrado realizado al conjunto de datos inició con la exclusión de las observaciones relacionadas con la Unión Europea, debido a que no es un país sino un conjunto, lo que ocasiona problema de doble conteo. Asimismo, Kosovo se eliminó por la falta de información disponible en la librería *pycountry*. También se suprimió a Hong Kong del análisis, ya que mostraba un nivel atípicamente alto de *unique pushers* en comparación con otros países. El período de tratamiento comenzó en el cuarto trimestre de 2022, alineándose con el lanzamiento de ChatGPT, ocurrido el 30 de noviembre de ese mismo año.

Las unidades tratadas, en el presente trabajo, corresponden a los países que originalmente comunicaron que tendrían acceso a la aplicación ChatGPT. Por tanto, durante el período de tiempo indicado, no se añadieron nuevos países al grupo de tratamiento. Esto constituye un diseño de bloques, como se describe en Arkhangelsky et al. (2021), lo cual facilita el cálculo de los estimadores. El diseño del trabajo incluye 133 unidades de tratamiento y 32 unidades de control; 12 períodos de pretratamiento, 4 períodos de tratamiento y un total de 26400 observaciones.

4. Estadísticos descriptivos

En esta sección se presentan los estadísticos descriptivos fundamentales que caracterizan los datos utilizados en el estudio. Dichos datos, obtenidos del Innovation Graph de GitHub, incluyen información detallada sobre los *unique pushers* a nivel de lenguaje de programación y trimestre. El análisis de las principales tendencias y patrones observados en los datos proporciona un contexto claro y comprensivo para la interpretación de los resultados subsecuentes.

En la Figura 1, se muestra la distribución porcentual de los lenguajes de programación más usados en ciencia de datos en países con disponibilidad de ChatGPT durante el periodo 2020–2023. En este grupo, JavaScript incrementa su participación de 30.9% en 2020 a 33.1% en 2023, consolidando su liderazgo. Python pasa de 18.4% a 19.4% en el mismo periodo, evidenciando un crecimiento moderado, mientras que TypeScript registra la expansión relativa más notable, al aumentar de 6.1% a 9.7%. En contraste, Java disminuye de 11.5% a 9.7%, y Ruby presenta una caída más pronunciada, de 7.1% a 3.7%. En contraste, en la Figura 2, correspondiente a países sin disponibilidad de ChatGPT, si bien la estructura general es similar, se observan algunas diferencias cuantitativas relevantes. JavaScript se mantiene prácticamente estable, pasando de 30.5% en 2020 a 30.0% en 2023. Python experimenta un crecimiento más intenso que en el grupo tratado, aumentando de 15.7% a 18.4%. TypeScript también crece de forma marcada, de 4.5% a 9.6%,

replicando una tendencia similar a la observada en la Figura 1. Por su parte, Java presenta una reducción más acentuada, de 16.3% a 10.9%, y Ruby desciende de 6.0% a 2.9%.

La comparación entre la Tabla 1 y Tabla 2 evidencia diferencias relevantes entre el grupo de control y el grupo de tratamiento en términos de actividad de desarrollo medida por el número de *unique pushers*. En el periodo pretratamiento, el grupo tratado presenta medias superiores en la mayoría de los lenguajes, por ejemplo, JavaScript (51.00 frente a 18.84) y Python (22.71 frente a 13.30), diferencia que se mantiene en el periodo postratamiento (58.98 vs. 34.29 y 33.46 vs. 22.21, respectivamente). En ambos grupos los valores mínimos permanecen en 0.00, lo que evidencia heterogeneidad en la participación entre países, mientras que los valores máximos y las desviaciones estándar aumentan tras el periodo de análisis, especialmente en lenguajes con mayor dinamismo, indicando una expansión general pero desigual de la actividad en GitHub. Estos patrones sugieren la presencia de tendencias temporales comunes junto con diferencias estructurales entre grupos, lo que respalda la aplicación de una metodología de diferencias en diferencias para estimar el efecto del acceso a ChatGPT sobre la actividad de desarrollo en GitHub.

En la Figura 3, correspondiente a países con disponibilidad de ChatGPT, se observa un crecimiento sostenido en el número de *unique pushers* entre 2020 y 2023, con una aceleración más evidente a partir de 2022. JavaScript concentra los niveles más altos de actividad durante todo el periodo, incrementándose desde aproximadamente 3,200 desarrolladores en 2020-Q1 hasta cerca de 6,100 en 2023-Q4. Por su parte, Python también muestra un crecimiento considerable, pasando de alrededor de 1,800 a aproximadamente 3,600 desarrolladores en el mismo intervalo. Asimismo, TypeScript presenta una expansión notable, aumentando desde aproximadamente 600 desarrolladores al inicio del periodo hasta cerca de 2,100 en 2023-Q4, lo que evidencia una rápida adopción en los últimos años. En contraste, lenguajes como Java muestran incrementos más moderados, creciendo aproximadamente de 1,200 a cerca de 1,700 desarrolladores.

En la Figura 4 correspondiente a países sin disponibilidad de ChatGPT, también se identifica una tendencia creciente, aunque con valores absolutos menores y una evolución más gradual. En este caso, JavaScript aumenta desde aproximadamente 1,400 desarrolladores en 2020-Q1 hasta cerca de 3,500 en 2023-Q4. Python, por su parte, pasa de alrededor de 800 a aproximadamente 2,350 desarrolladores durante el mismo periodo. De manera similar, TypeScript crece progresivamente desde cerca de 250 desarrolladores hasta aproximadamente 1,250 hacia el final del periodo analizado. En el caso de Java, el incremento resulta más moderado, evolucionando desde aproximadamente 600 hasta cerca de 950–1,000 desarrolladores.

En conjunto, las cuatro figuras evidencian patrones similares en la evolución del ecosistema de desarrollo entre países con y sin disponibilidad de ChatGPT, aunque con diferencias en la magnitud del crecimiento observado. La Figura 1 y Figura 2 muestran que la popularidad de los principales lenguajes utilizados en ciencia de datos se mantiene dominada por JavaScript y Python en ambos grupos, mientras que TypeScript presenta el mayor incremento relativo durante el periodo analizado. Sin embargo, al considerar los valores absolutos presentados en las Figura 3 y Figura 4, se observa que el aumento en el número de *unique pushers* es más pronunciado en los países con disponibilidad de ChatGPT, especialmente a partir del cuarto trimestre de 2022, coincidiendo con el lanzamiento público de la herramienta. En contraste, lenguajes más tradicionales como Java y Ruby muestran una pérdida relativa de participación o un crecimiento más moderado en ambos contextos.

5. Metodología

Los datos de cada variable pueden ser representados como una matriz de $N \times T$, donde N son las unidades observadas a lo largo de T periodos de tiempo. Existen N_0 unidades de control y $N_1 = N - N_0$ unidades de tratamiento. De manera similar, se tienen T_0 periodos antes del tratamiento y $T_1 = T - T_0$ periodos después del tratamiento. Los modelos empleados generalmente se describen como casos específicos de la siguiente ecuación:

$$\widehat{ATE} = \frac{1}{N_1} \sum_{i=N_0+1}^N \left(\frac{1}{T_1} \sum_{t=T_0+1}^T Y_{it} - \frac{1}{T_0} \sum_{t=1}^{T_0} \lambda_t Y_{it} \right) - \sum_{i=1}^{N_0} \omega_i \left(\frac{1}{T_1} \sum_{t=T_0+1}^T Y_{it} - \frac{1}{T_0} \sum_{t=1}^{T_0} \lambda_t Y_{it} \right)$$

Donde Y_{it} es la variable de resultado para la unidad i en el periodo de tiempo t ; ω_i es el factor de ponderación para la unidad i , tal que $\sum_{i=1}^{N_0} \omega_i = 1$; y λ_t es el factor de ponderación para el periodo de tiempo t , tal que $\sum_{t=1}^{T_0} \lambda_t = 1$. El primer término en el lado derecho representa la diferencia entre el nivel promedio del resultado antes y después del tratamiento para el grupo de tratamiento. El segundo término del lado derecho mide lo mismo, pero para el grupo de control. La diferencia entre cada modelo radica en la forma en que se calculan los pesos de las unidades y del tiempo.

5.1. Diferencia en Diferencias (DiD)

El modelo DiD se aplica en casos que exista un número considerable de unidades expuestas al programa y se centra en estimar el efecto de la exposición al tratamiento resolviendo la regresión de efectos fijos sin ponderaciones temporales ni unitarias. Dicho método se emplea para estimar el efecto del lanzamiento de ChatGPT en la actividad de desarrollo de software orientado en la ciencia de datos, comparando el comportamiento de esta actividad antes y después del lanzamiento de chatbot con un grupo de control que no fue afectado por esta intervención. Arkhangelsky (2021) menciona que los supuestos que debe cumplir el modelo DiD incluyen las tendencias paralelas; es decir, las trayectorias del grupo tratado y del grupo de control deben ser idénticas a lo largo del tiempo, en ausencia de tratamiento. Además, se asume que el programa no produce efectos antes de su implementación; en otras palabras, los efectos del tratamiento son homogéneos dentro del grupo de tratamiento y que no hay otros eventos indistintamente a los grupos durante el mismo período. Asimismo, existe un supuesto menos estricto en la asignación aleatoria de las unidades tratadas. Esto facilita su aplicación, la validación de otros supuestos y mejorando así la robustez de la estimación del modelo.

$$\omega_i = \frac{1}{N_0}; \forall i = 1, \dots, N_0$$

$$\lambda_t = \frac{1}{T_0}; \forall t = 1, \dots, T_0$$

5.2. Control Sintético (CS)

El método de CS se aplica en entornos con una sola o pocas unidades expuestas al tratamiento y su objetivo es compensar la falta de tendencias paralelas a través de

la ponderación de las unidades para que coincidan con sus tendencias previas a la exposición. Dicho modelo se utiliza para crear un contrafactual creíble de la unidad tratada mediante la ponderación de unidades de control. Arkhangelsky (2021) expone los supuestos que deben existir en el método CS, que incluyen la capacidad de ponderar unidades de control o no tratadas para alinear sus tendencias pretratamiento con las de las unidades tratadas. Estas ponderaciones deben permanecer constantes en el tiempo, no debe haber efectos de tratamientos no observados que tengan un impacto significativo y una asignación de pesos adecuada debe garantizar un ajuste adecuado en el período pretratamiento. Se supone que las ponderaciones temporales equilibran los tiempos previos y posteriores al tratamiento.

El vector $(\omega_1, \dots, \omega_{N_0})$ es definido como:

$$(\omega_1, \dots, \omega_{N_0}) = \arg \min_{\omega_1, \dots, \omega_{N_0}} \sum_{t=1}^{T_0} \left(\sum_{i=1}^{N_0} \omega_i Y_{it} - \frac{1}{N_1} \sum_{i=N_0+1}^N Y_{it} \right)^2$$

Sujeto a:

$$\sum_{i=1}^{N_0} \omega_i = 1; \quad \omega_i \geq 0; \quad \forall i = 1, \dots, N_0$$

Por otro lado, el método CS se enfoca únicamente en los promedios no ponderados de los periodos de tratamiento, sin considerar la diferencia con los periodos anteriores al tratamiento. Esto implica que $\lambda_t = 0$ para todos los periodos previos al tratamiento.

5.3. Diferencia en Diferencias Sintética (SDiD)

El modelo de DiDS combina las técnicas de DiD y CS para mejorar la precisión de la estimación del efecto del tratamiento, creando y asegurando el supuesto de tendencias paralelas en caso no se cumplan. Según Arkhangelsky (2021), incluyen la capacidad de ponderar unidades de control o no tratadas para alinear sus tendencias pretratamiento con las de las unidades tratadas, estas ponderaciones permanecen constantes en el tiempo; es decir, no hay efectos de tratamientos no observados que tengan un impacto significativo, y una asignación de pesos adecuada garantiza un ajuste adecuado en el período pretratamiento. Se supone que las ponderaciones temporales equilibran los tiempos previos y posteriores al tratamiento.

En este contexto, se considera el vector $\omega_0, \omega_1, \dots, \omega_N$, el cual incluye los factores de ponderación para las unidades de tratamiento y ω_0 , un término de intersección. Estos se definen de la siguiente manera:

$$\omega_i = \frac{1}{N_1}; \quad \forall i = N_0 + 1, \dots, N$$

$$(\omega_0, \dots, \omega_N) = \arg \min_{\omega_0, \dots, \omega_N} \sum_{t=1}^{T_0} \left(\omega_0 + \sum_{i=1}^{N_0} \omega_i Y_{it} - \frac{1}{N_1} \sum_{i=N_0+1}^N Y_{it} \right)^2 + \zeta^2 T_0 \|(\omega_0, \omega_1, \dots, \omega_N)\|_2^2$$

Sujeto a:

$$\sum_{i=1}^{N_0} \omega_i = 1; \omega_i \geq 0; \forall i = 1, \dots, N_0$$

El término ubicado más a la derecha corresponde a la regularización, donde:

$$\zeta = (N_1 T_1)^{1/4} \hat{\sigma}$$

$$\hat{\sigma}^2 = \frac{1}{N_0(T_0 - 1)} \sum_{i=1}^{N_0} \sum_{t=1}^{T_0-1} \Delta_{it}$$

Arkhangelsky et al. (2021) describen cómo este factor de regularización es útil para considerar los resultados que están correlacionados a nivel de unidad y en el tiempo. En cambio, se supone que la correlación entre unidades y a lo largo del tiempo se captura mediante variaciones sistemáticas en los resultados. Por lo tanto, la regularización no se aplica en la estimación de λ_i . Para este modelo, el vector $(\lambda_0, \dots, \lambda_{T_0})$ se define de la siguiente forma:

$$(\lambda_0, \dots, \lambda_{T_0}) = \arg \min_{\lambda_0, \dots, \lambda_{T_0}} \sum_{i=1}^{N_0} \left(\lambda_0 + \sum_{t=1}^{T_0} \lambda_t Y_{it} - \frac{1}{T_1} \sum_{t=T_0+1}^T Y_{it} \right)^2$$

Sujeto a:

$$\sum_{t=1}^{T_0} \lambda_t = 1; \lambda_t \geq 0; \forall t = 1, \dots, T_0$$

El estimador SDID da más peso a las unidades que, en promedio, son similares en términos de pasado a las unidades tratadas (objetivo) y a los períodos que, en promedio, son similares a los períodos tratados (objetivo). En ese sentido, se dice que el uso de ponderaciones en el estimador SDID hace hincapié en las unidades y en los períodos para un modelo más adecuado. Así, este método SDiD cuenta con una gran ventaja en su regulación, porque penaliza la optimización de los pesos asignados a las unidades de control y periodos de tiempo. Al limitar la suma de los cuadrados de los pesos, se reduce el sesgo y se evita la sobreestimación, lo que asegura una mayor dispersión y soluciones auténticas. El parámetro de regularización (ζ) se ajusta en función de la variabilidad en los cambios del pretratamiento. Por lo tanto, SDiD mejora la precisión y la robustez del estimador.

5.4. Inferencia

Los datos empleados en este estudio son exclusivos, puesto que permiten incluir un gran número de unidades tratadas, todas expuestas de manera simultánea. Esta característica brinda la oportunidad de aplicar el estimador *bootstrap*, que Arkhangelsky et al. (2021) demostraron ser asintóticamente normal cuando se cuenta con un gran número de unidades tratadas. Para el diseño del algoritmo *bootstrap* destinado a estimar el error estándar de los estimadores del ATE, se remite a Clarke et al. (2023).

6. Resultados

En esta sección se presentan los resultados del análisis estadístico sobre el impacto de la disponibilidad de ChatGPT en la actividad de desarrollo de software, enfocándonos en los diez lenguajes de programación más utilizados. Para este análisis, hemos empleado tres enfoques metodológicos robustos: Diferencia en Diferencias, Control Sintético y Diferencia en Diferencias Sintéticas. El objetivo es medir el número de *unique pushers* por cada 100 mil habitantes y evaluar las diferencias entre los países con acceso a ChatGPT y aquellos sin acceso. Los resultados, desglosados por lenguaje de programación y país, proporcionan una perspectiva detallada de cómo ChatGPT ha influido en la productividad de los desarrolladores a lo largo del tiempo. A continuación, se presentan las conclusiones clave, ilustradas mediante figuras comparativas y tablas de estadísticas descriptivas.

La Tabla 3 presenta el impacto de la disponibilidad de ChatGPT en el desarrollo de software respecto a cada lenguaje de programación. Para el lenguaje C, la disponibilidad de ChatGPT produjo un incremento significativo de 2,680 *unique pushers* (error estándar de 0,539) según la estimación DiD, con aumentos adicionales observados en CS y SDiD (1,478 y 1,241 *unique pushers*, respectivamente). En C#, aunque la estimación DiD muestra un incremento de 1,503 *unique pushers* (error estándar de 0.382), los métodos CS y SDiD no presentan significancia estadística en sus coeficientes (0,992 y 0,446). Para C++, se reporta un aumento de 2,202 *unique pushers* según la estimación DiD (error estándar de 0,460), con incrementos también presentes en CS y SDiD (1,173 y 1,118 *unique pushers*, respectivamente). En Go, la disponibilidad de ChatGPT resultó en un aumento de 1,383 *unique pushers* (error estándar de 0,347) según la estimación DiD, con incrementos también observados en CS y SDiD (0,668 y 0,702 *unique pushers*), considerando que la estimación de CS no fue significativa. En Java, se registra un incremento de 2,774 *unique pushers* (error estándar de 0.670) según DiD, con aumentos en CS y SDiD (2,159 y 1,687, aunque el estimador de CS no es significativo). Para JavaScript, se evidencia un aumento de 13,557 *unique pushers* (error estándar de 1,973) según DiD, con incrementos en CS y SDiD (9,198 y 8,116 *unique pushers*). En PHP, se observa un aumento de 0,895 *unique pushers* (error estándar de 0,358) según DiD, con incrementos en CS y SDiD (0,990 y 0,914, aunque el estimador de CS no es significativo). Python muestra un incremento de 7,664 *unique pushers* (error estándar de 1,460) según DiD, con aumentos en CS y SDiD (1,619 y 4,431, aunque en el estimador de CS no es significativo). En Ruby, aunque las estimaciones DiD y CS no son significativas (0.243 y 1.488 *unique pushers*, errores estándar de 0.372 y 0,976), el método y SDiD muestra un incremento significativo (0,795 *unique pushers*). Finalmente, en TypeScript, se registra un incremento significativo de 7,115 *unique pushers* (error estándar de 1,192) según DiD, con aumentos en CS y SDiD (3,205 y 2,994 *unique pushers*, aunque el estimador de CS no es significativo).

Por otra parte, de la Figura 5 - Figura 14, se muestran las tendencias y ponderaciones estimadas de *unique pushers* por cada 100 mil personas para diferentes lenguajes de programación, utilizando los métodos Diferencia en Diferencias (DiD), Control Sintético (CS) y Diferencia en Diferencias Sintética (SDiD). Cada figura se divide en seis paneles: los paneles (a) y (b) presentan los resultados del método DiD, los paneles (c) y (d) muestran los resultados del método CS, y los paneles (e) y (f) ilustran los resultados del método SDiD.

En cuanto al ajuste y validación del contrafactual, el método DiD, como se observa en los paneles (a), muestra divergencia en las tendencias para el grupo tratado y el

grupo de control antes del tratamiento, indicando que la suposición de tendencias paralelas no se cumple, llevando a estimaciones sesgadas del impacto del tratamiento. Por otro lado, el método CS, ilustrado en los paneles (c), muestra tendencias más similares antes del tratamiento, sugiriendo un mejor ajuste del contrafactual. Sin embargo, la falta de ponderación temporal puede afectar la precisión post-tratamiento. El método SDiD, representado en los paneles (e), muestra tendencias muy similares antes del tratamiento, validando así la construcción del contrafactual. El área sombreada en estos paneles indica la ponderación temporal, destacando la importancia relativa de cada periodo pre-tratamiento y mejorando la precisión del ajuste. La divergencia clara entre las líneas después del tratamiento sugiere un impacto significativo del tratamiento, indicando que SDiD ha proporcionado un ajuste preciso y confiable.

En el método DiD, representado en los paneles (b), todos los estados de control reciben el mismo peso, lo cual es indicado por la línea roja horizontal constante. Este método no considera la heterogeneidad entre las unidades de control, lo que puede ser una limitación significativa si las tendencias previas al tratamiento no son paralelas. Por otro lado, el método CS, ilustrado en los paneles (d), asigna pesos específicos a las unidades de control, permitiendo una mayor flexibilidad y una mejor adaptación a las características específicas de las unidades de control. La línea roja horizontal en estos paneles refleja la media ponderada de los pesos asignados, que puede cambiar dependiendo de la distribución de los pesos. En CS, las cruces representan los pesos igual a 0 y los puntos representan pesos mayores a cero, con el tamaño de los puntos indicando la magnitud del peso. El método SDiD, mostrado en los paneles (f), asigna pesos variables tanto a las unidades de control como a los periodos de tiempo, proporcionando una asignación de pesos más robusta y flexible. La línea roja horizontal en estos paneles también refleja la media ponderada de los pesos, cambiando significativamente debido a la ponderación adicional por el tiempo. En SDiD, las cruces representan los pesos igual a 0 y los puntos representan pesos mayores a cero, con el tamaño de los puntos indicando la magnitud del peso.

Al comparar la Figura 5 y la Figura 13 (C y Ruby), se observa que Ruby muestra una mayor dispersión y variabilidad en la asignación de pesos en comparación con C, y las tendencias pre-tratamiento no cumplen con la suposición de paralelismo tan bien como en C. En Ruby, las cruces que indican pesos igual a 0 son más frecuentes, sugiriendo que varias unidades de control no contribuyen al contrafactual. Una mayor variabilidad de pesos implica que algunas unidades de control tienen una influencia mucho mayor en la construcción del contrafactual, lo cual puede reflejar una mayor heterogeneidad en las características del grupo de control. Al comparar las Figura 9 y Figura 12 (Java y Python), se nota que Python presenta una mayor variabilidad en la asignación de pesos y una divergencia post-tratamiento más marcada. Los puntos en Python indican una mayor heterogeneidad en las unidades de control en comparación con Java. Una menor variabilidad de pesos, como se observa en Java, sugiere que las unidades de control son más homogéneas y que cada una contribuye de manera más equitativa a la construcción del contrafactual. Asimismo, la Figura 6 (C#), Figura 7 (C++), Figura 8 (Go), Figura 10 (JavaScript), Figura 11 (PHP) y Figura 14 (TypeScript) muestran tendencias pre y post-tratamiento relativamente paralelas y homogéneas. Las asignaciones de pesos en CS y SDiD son bastante similares, con poca variabilidad y pocas cruces representando pesos igual a 0. Esto sugiere que, para estos lenguajes, las unidades de control son bastante homogéneas y la ponderación temporal no muestra grandes variaciones.

7. Conclusiones y recomendaciones de política

En conclusión, los resultados obtenidos mediante el modelo SDID indican que la disponibilidad de ChatGPT en la mayoría de los países del mundo ha tenido un impacto positivo y diferenciado en el desarrollo de software en varios lenguajes de programación. Específicamente, se observó un aumento notable en la cantidad de desarrolladores que utilizan lenguajes como JavaScript, Python, Java y TypeScript. Estos hallazgos sugieren que la asistencia proporcionada por ChatGPT motiva el desarrollo de software en estos lenguajes. Por otro lado, lenguajes como C# no mostraron un impacto significativo, lo que sugiere la necesidad de investigaciones adicionales para entender las razones detrás de esta disparidad. Podría ser que factores específicos del lenguaje o del contexto de uso influyan en la efectividad de ChatGPT. Este impacto sobre el desarrollo de software muestra variaciones significativas entre diferentes contextos nacionales también resalta que el impacto de esta herramienta no es uniforme. Factores como la infraestructura tecnológica, la adopción de tecnologías y las políticas de apoyo al desarrollo de software pueden desempeñar un papel crucial en la determinación de la magnitud del impacto.

Por tanto, aunque ChatGPT no constituye un asistente de codificación perfecto y presenta desafíos en cuanto a precisión y calidad del código generado, la evidencia muestra que su disponibilidad ha contribuido de manera significativa al crecimiento del número de desarrolladores activos en el campo de la ciencia de datos. Esta herramienta ha facilitado la entrada de nuevos programadores, ha acelerado la resolución de problemas comunes y ha promovido una mayor participación en plataformas colaborativas como GitHub. Así, su impacto positivo en el ecosistema de desarrollo justifica una discusión profunda sobre su integración responsable en los flujos de trabajo tecnológicos y su papel en la configuración futura del trabajo en la economía digital.

En línea con los resultados obtenidos, una primera recomendación de política consiste en dotar de acceso gratuito a versiones avanzadas de ChatGPT para desarrolladores junior orientados a ciencia de datos dentro de empresas vinculadas al sector tecnológico. Esta medida podría implementarse mediante incentivos para que las empresas inviertan en la adquisición de suscripciones a la versión premium de la herramienta por periodos iniciales de seis meses renovables. La evidencia sugiere que los desarrolladores juniors utilizan ChatGPT principalmente para resolver consultas básicas y superar barreras iniciales en el proceso de programación, mientras que, a medida que adquieren mayor experiencia, continúan utilizándolo para acelerar su productividad. Asimismo, las versiones avanzadas del modelo presentan mayores capacidades de asistencia en programación, lo que puede facilitar el desarrollo de tareas técnicas y mejorar la integración de nuevos programadores en entornos laborales reales. El periodo de seis meses permitiría evaluar razonablemente la evolución del desempeño del trabajador, por lo que se recomienda establecer criterios de elegibilidad verificables, como la exigencia de cuentas de GitHub validadas y con actividad reciente, permitiendo observar la generación de código a lo largo del tiempo y asegurar que el beneficio se dirija efectivamente a perfiles junior.

De manera complementaria, se recomienda implementar incentivos a la productividad asociados al uso de herramientas de inteligencia artificial dentro de las empresas del sector software, mediante la asignación de bonos económicos a trabajadores que evidencien mejoras en su rendimiento atribuibles al uso de estas tecnologías. El monto del incentivo podría ser determinado por cada empresa según sus objetivos y capacidades internas. Para garantizar una evaluación objetiva, el

acceso a este beneficio debería sustentarse en mecanismos de seguimiento bimensuales que consideren indicadores como la cantidad de horas de uso de herramientas de inteligencia artificial, el volumen de tareas completadas con su asistencia y el cumplimiento de estándares mínimos de calidad establecidos por los equipos de aseguramiento de calidad (Quality Assurance, QA). De esta forma, ambas medidas contribuirían a fomentar una adopción productiva y responsable de la inteligencia artificial, fortaleciendo la participación de desarrolladores en el ecosistema tecnológico y promoviendo mejoras medibles en el desempeño laboral.

8. Tablas

Tabla 1

Estadísticos descriptivos del número de unique pushers por lenguaje de programación en países del grupo de control (periodos pre y postratamiento)

Lenguaje	Pretratamiento				Postratamiento			
	Media	Mínimo	Máximo	Desv. Est.	Media	Mínimo	Máximo	Desv. Est.
C	4.88	0.00	210.10	22.41	8.73	0.00	215.00	36.00
C#	3.01	0.00	61.62	8.33	4.41	0.00	60.82	11.48
C++	5.28	0.00	211.28	22.82	8.84	0.00	216.88	36.25
Go	1.74	0.00	149.67	12.29	4.85	0.00	142.99	24.10
Java	6.59	0.00	271.23	25.82	10.81	0.00	233.03	39.55
JavaScript	18.84	0.00	640.90	63.46	34.29	0.00	605.37	102.79
PHP	2.27	0.00	63.75	7.88	3.46	0.00	48.18	8.36
Python	13.30	0.00	454.19	49.96	22.21	0.00	482.93	79.05
Ruby	3.13	0.00	108.09	12.00	4.00	0.00	102.32	17.21
TypeScript	3.46	0.00	224.19	17.55	11.79	0.00	275.74	45.14

Nota. Estadísticos descriptivos (media, mínimo, máximo y desviación estándar) para países del grupo control en periodos pre y post tratamiento. Elaboración propia con datos de GitHub (2020–2023).

Tabla 2

Estadísticos descriptivos del número de unique pushers por lenguaje de programación en países del grupo de tratamiento (periodos pre y postratamiento)

Lenguaje	Pretratamiento				Postratamiento			
	Media	Mínimo	Máximo	Desv. Est.	Media	Mínimo	Máximo	Desv. Est.
C	7.16	0.00	82.05	10.91	10.91	0.00	117.10	14.88
C#	7.39	0.00	62.97	10.45	9.64	0.00	65.53	12.53
C++	7.77	0.00	85.60	11.64	10.82	0.00	115.69	15.10
Go	3.63	0.00	47.68	6.83	4.05	0.00	76.18	7.91
Java	13.58	0.00	118.42	16.28	16.14	0.00	156.79	20.07
JavaScript	51.00	0.00	348.52	56.30	58.98	0.00	455.33	62.87
PHP	7.54	0.00	39.27	8.29	8.24	0.00	39.27	8.42
Python	22.71	0.00	270.36	33.47	33.46	0.00	365.32	46.23
Ruby	6.07	0.00	84.47	10.00	6.28	0.00	84.47	10.01
TypeScript	17.32	0.00	181.23	21.61	18.83	0.00	181.23	23.56

Nota. Estadísticos descriptivos (media, mínimo, máximo y desviación estándar) para países del grupo tratado en periodos pre y post tratamiento. Elaboración propia con datos de GitHub (2020–2023).

Tabla 3

Impacto heterogéneo de ChatGPT en el desarrollo de software vinculado a Ciencia de Datos

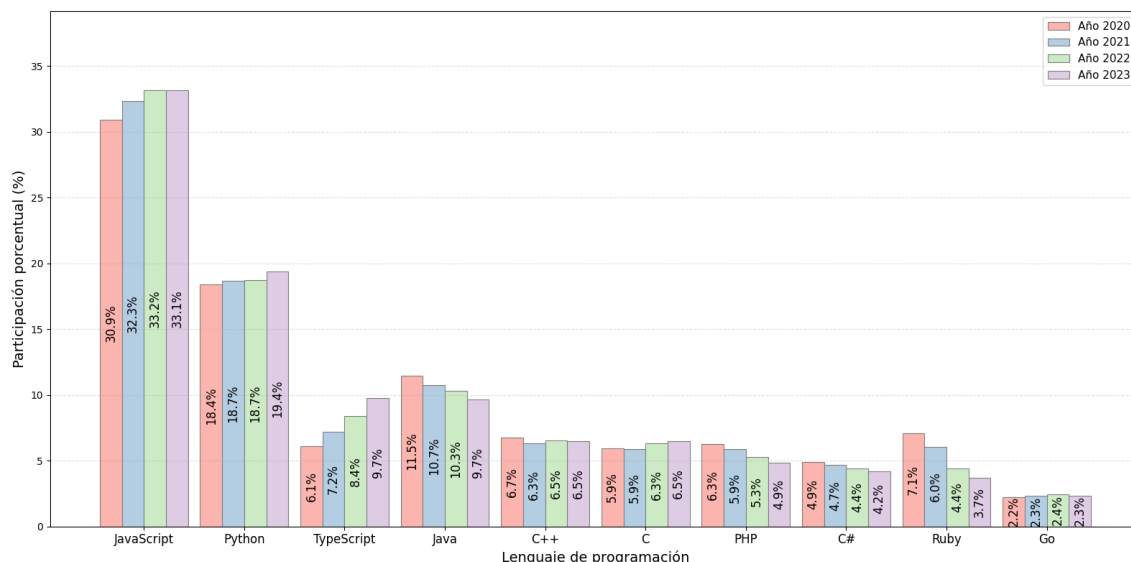
Lenguaje	DID	SC	SDID	Obs.	Baseline Mean
C	2.680** (0.539)	1.478 (0.978)	1.241** (0.443)	2624	1.252
C#	1.503** (0.382)	0.992 (1.262)	0.446 (0.383)	2624	1.901
C++	2.202** (0.460)	1.173 (0.874)	1.118** (0.385)	2624	1.558
Go	1.383** (0.347)	0.668 (0.656)	0.702** (0.240)	2624	0.438
Java	2.774** (0.670)	2.159 (2.044)	1.687* (0.911)	2624	3.432
JavaScript	13.557** (1.973)	9.198* (4.950)	8.116** (2.732)	2624	11.84
PHP	0.895* (0.358)	0.990 (0.921)	0.914* (0.434)	2624	1.912
Python	7.664** (1.460)	1.619 (3.641)	4.431** (1.139)	2624	5.147
Ruby	0.243 (0.372)	1.488 (0.976)	0.795** (0.261)	2624	1.278
TypeScript	7.115** (1.192)	3.205 (2.306)	2.994** (0.923)	2624	1.923

Nota. Errores estándar entre paréntesis. * $p < 0.10$, ** $p < 0.05$, *** $p < 0.01$

9. Figuras

Figura 1

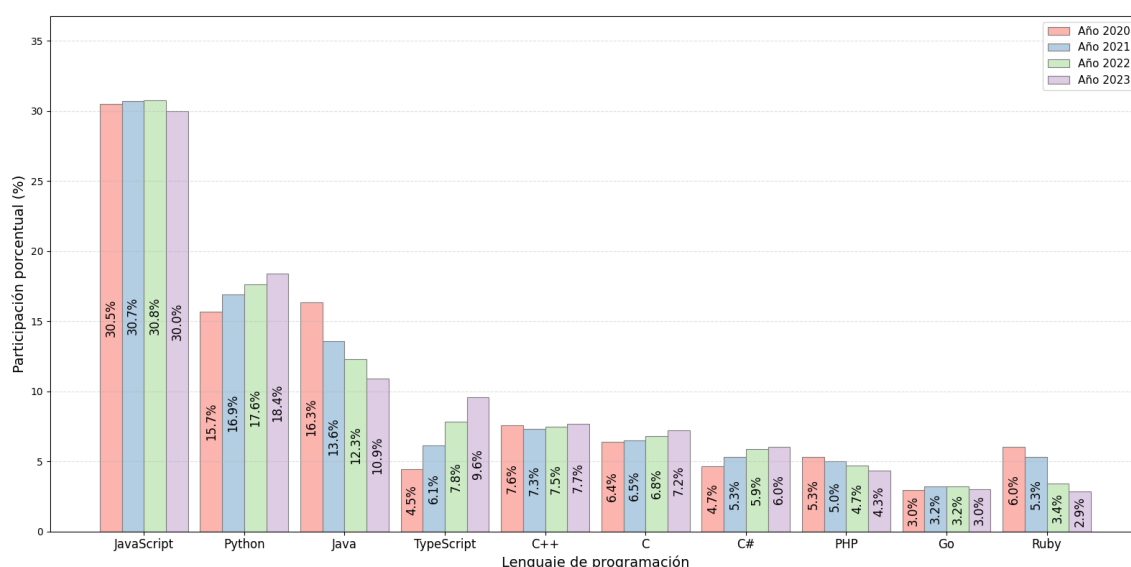
Distribución de lenguajes de programación más populares en países con ChatGPT disponible (2020-2023)



Nota. La figura presenta una distribución porcentual de la participación de los lenguajes más usados en ciencia de datos en los países cuyo acceso a ChatGPT está vigente. La base de datos proviene de GitHub Innovation Graph y, para este caso, se han usado un total de 21280 observaciones. Por otra parte, la cantidad de lenguajes de programación filtrados y usados son 10 y el número de años incluidos es 4. Respecto a la figura, se puede identificar un aumento constante de los lenguajes JavaScript, Python y TypeScript entre 2020 y 2023, siendo los más usados por los desarrolladores en esos períodos de tiempo, mientras que C#, Ruby y Go resaltan por ser los menos utilizados en el mismo segmento temporal. Elaboración propia con datos de GitHub (2020–2023).

Figura 2

Distribución de lenguajes de programación más populares en países sin ChatGPT disponible (2020-2023)

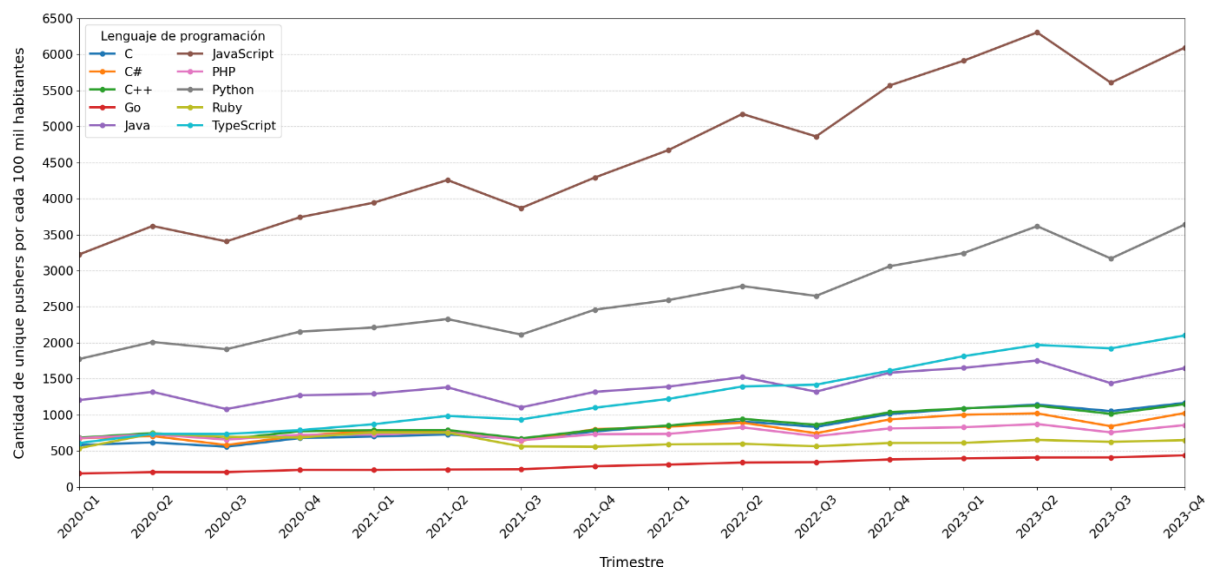


Nota. La figura presenta una distribución porcentual de la participación de los lenguajes más usados en ciencia de datos en los países cuyo acceso a ChatGPT está restringido. La base de datos proviene de GitHub Innovation Graph y, para este caso, se han usado un total de 5120 observaciones. Asimismo,

la cantidad de lenguajes de programación filtrados y usados son 10 y el número de años incluidos es 4. Respecto a la figura, se puede identificar un aumento casi constante de los lenguajes JavaScript y Python entre 2020 y 2023, siendo los más usados por los desarrolladores en esos períodos de tiempo, mientras que PHP, Go y Ruby destacan por ser los menos utilizados en el mismo segmento temporal. Elaboración propia con datos de GitHub (2020–2023).

Figura 3

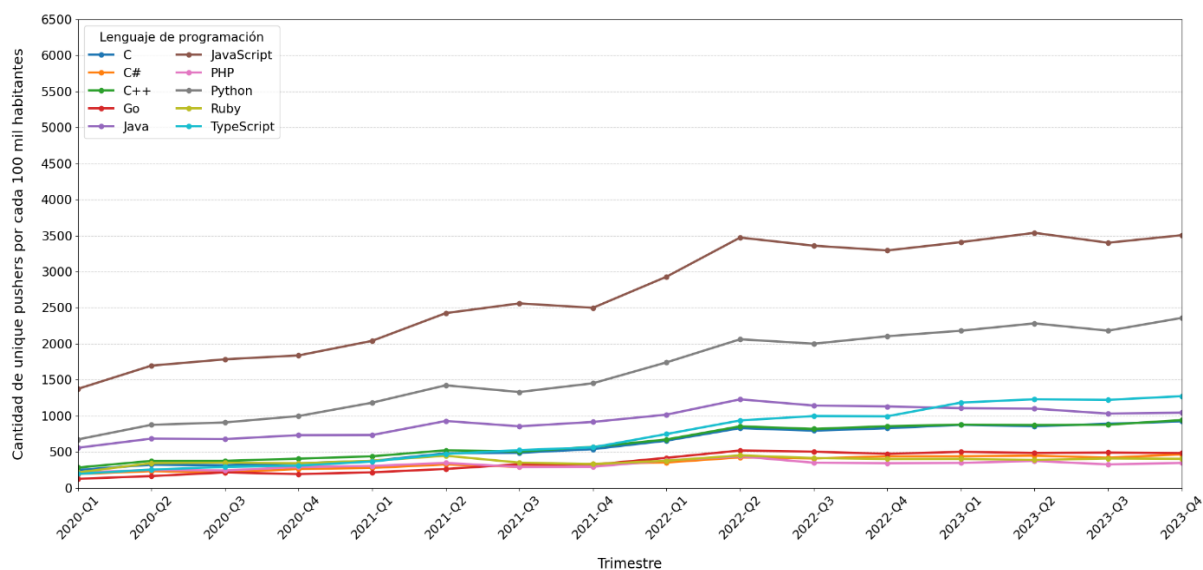
Distribución de unique pushers según lenguaje en países con ChatGPT (2020–2023)



Nota. La figura muestra una serie temporal sobre la dinámica de los unique pushers de cada lenguaje de programación en país cuyo acceso a ChatGPT está disponible. La base de datos proviene de GitHub Innovation Graph y, para este caso, se han usado un total de 160 observaciones. Además, la cantidad de lenguajes de programación filtrados y usados son 10 y el número de trimestres analizados es 16. Sobre la figura, se puede observar un aumento alto y progresivo del número de desarrolladores que al menos han hecho un push en GitHub por cada 100 mil personas para los lenguajes de JavaScript, Python y Java, siendo los dos primeros con mayor representación. Los lenguajes restantes obtuvieron crecimientos muy ligeros o se mantuvieron constantes a lo largo del período en cuestión. Elaboración propia con datos de GitHub (2020–2023).

Figura 4

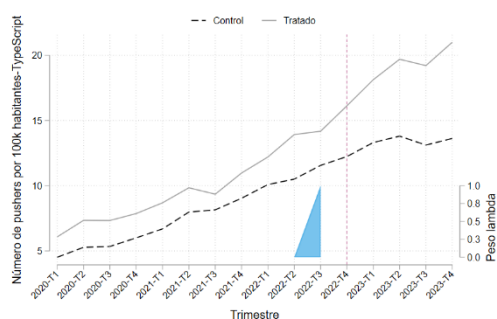
Distribución de unique pushers según lenguaje en países sin ChatGPT (2020–2023)



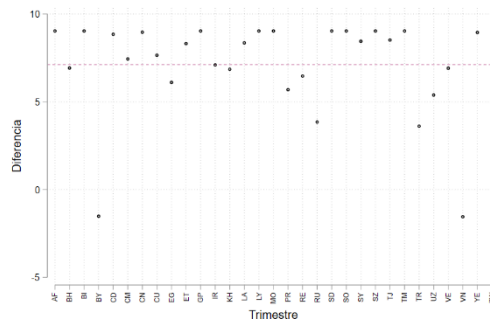
Nota. La figura muestra una serie temporal sobre la dinámica de los unique pushers de cada lenguaje de programación en país cuyo acceso a ChatGPT está restringido. La base de datos proviene de GitHub Innovation Graph y, para este caso, se han usado un total de 160 observaciones. Además, la cantidad de lenguajes de programación filtrados y usados son 10 y el número de trimestres analizados es 16. Sobre la figura, se puede observar un aumento mediano y progresivo del número de desarrolladores que al menos han hecho un push en GitHub por cada 100 mil personas para los lenguajes de JavaScript, Python y TypeScript, siendo los dos primeros con mayor representación. Los lenguajes restantes obtuvieron crecimientos muy ligeros, se mantuvieron constantes o cayeron a lo largo del período en cuestión. Elaboración propia con datos de GitHub (2020–2023).

Figura 5

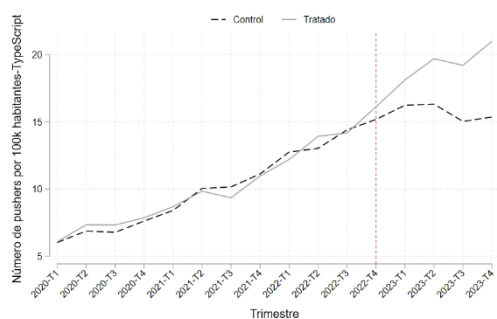
Tendencias y ponderaciones estimadas de TypeScript unique pushers por cada 100 mil personas



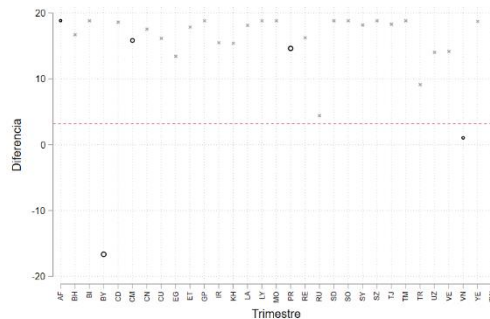
(a) Tendencias estimadas de DiD



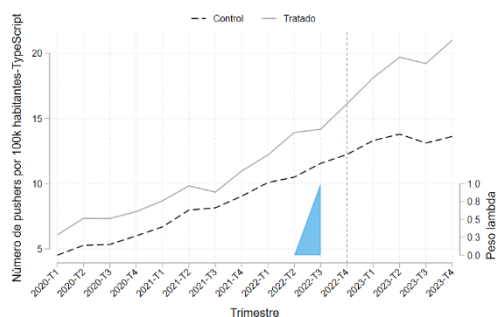
(b) Pesos DiD estimados



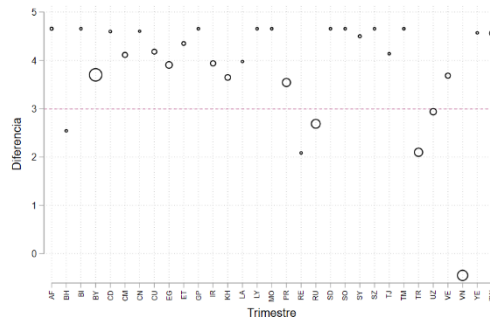
(c) Tendencias estimadas de CS



(d) Pesos CS estimados



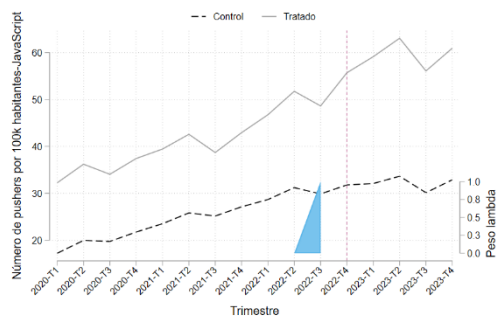
(e) Tendencias estimadas de SDiD



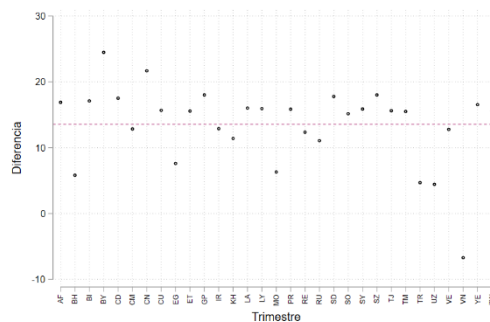
(f) Pesos SDiD estimados

Figura 6

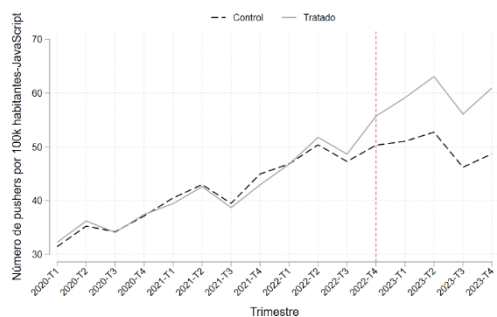
Tendencias y ponderaciones estimadas de JavaScript unique pushers por cada 100 mil personas



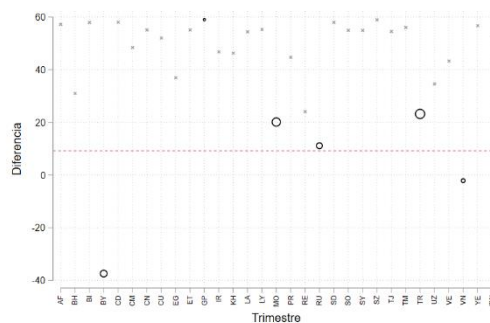
(a) Tendencias estimadas de DiD



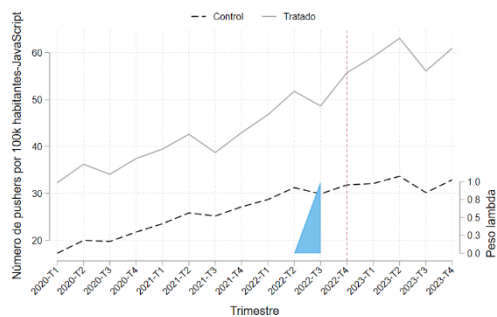
(b) Pesos DiD estimados



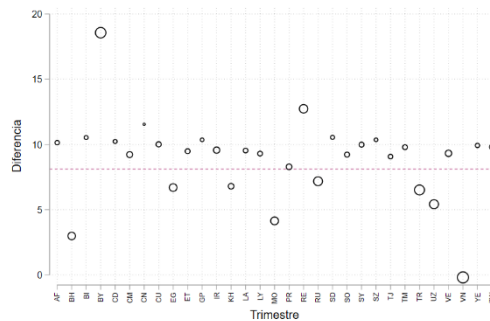
(c) Tendencias estimadas de CS



(d) Pesos CS estimados



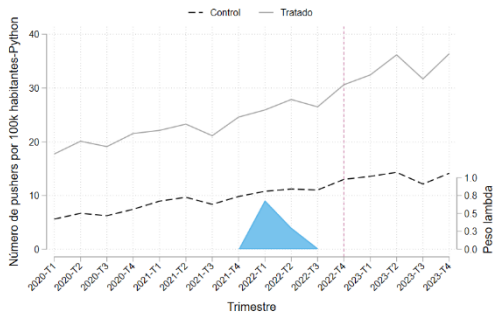
(e) Tendencias estimadas de SDiD



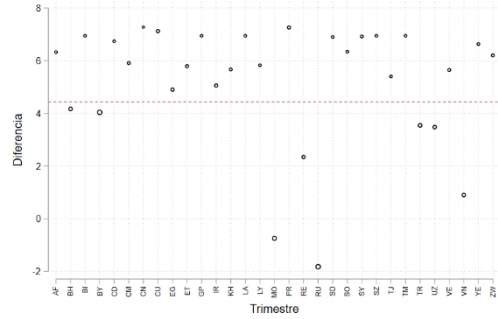
(f) Pesos SDiD estimados

Figura 7

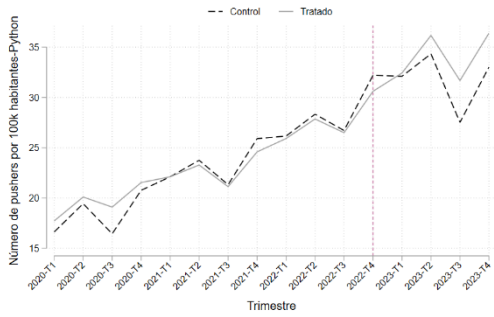
Tendencias y ponderaciones estimadas de Python unique pushers por cada 100 mil personas



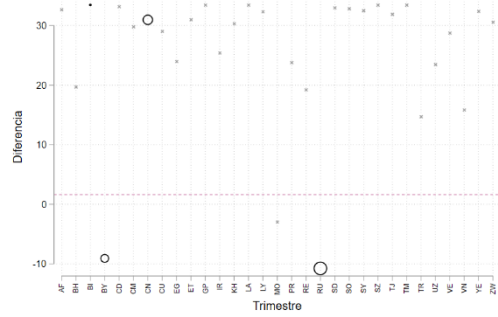
(a) Tendencias estimadas de DiD



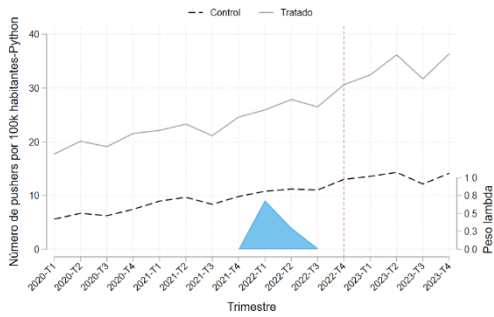
(b) Pesos DiD estimados



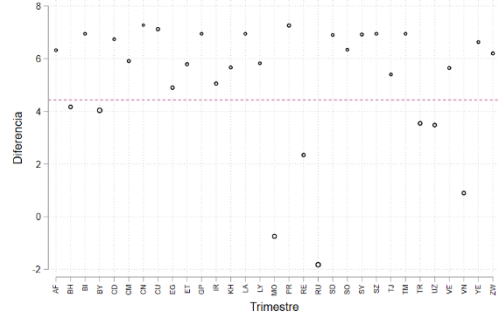
(c) Tendencias estimadas de CS



(d) Pesos CS estimados



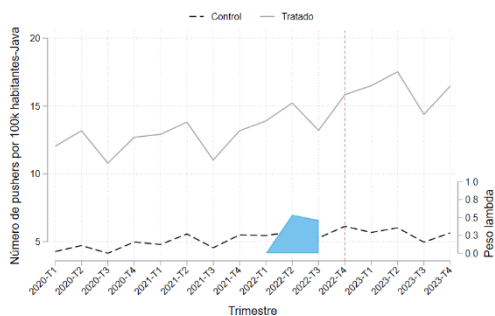
(e) Tendencias estimadas de SDiD



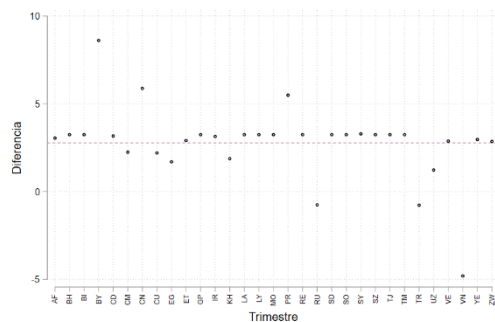
(f) Pesos SDiD estimados

Figura 8

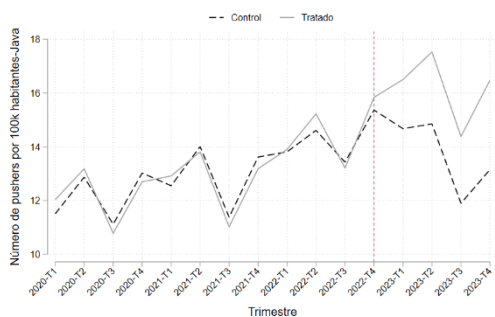
Tendencias y ponderaciones estimadas de Java unique pushers por cada 100 mil personas



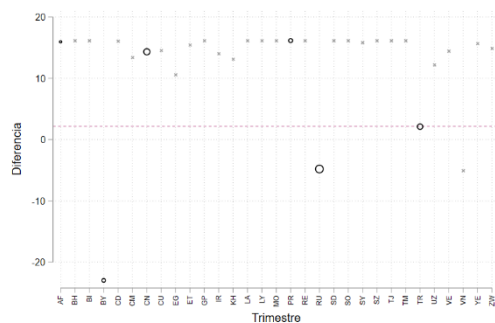
(a) Tendencias estimadas de DiD



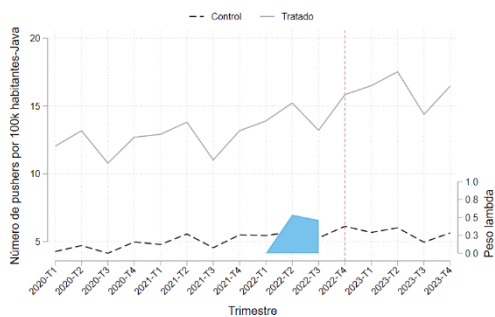
(b) Pesos DiD estimados



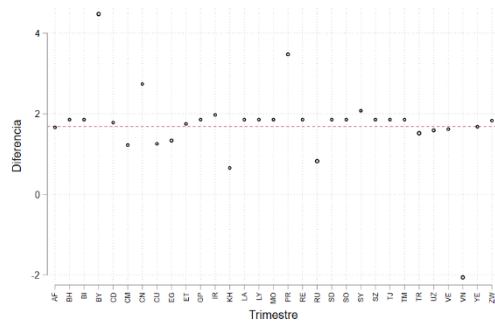
(c) Tendencias estimadas de CS



(d) Pesos CS estimados



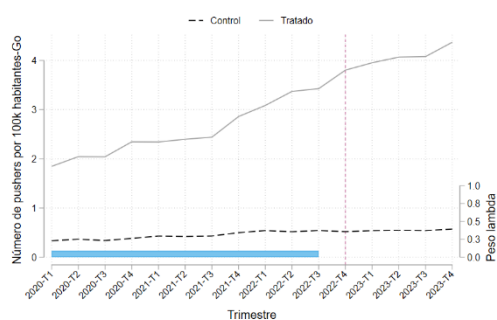
(e) Tendencias estimadas de SDiD



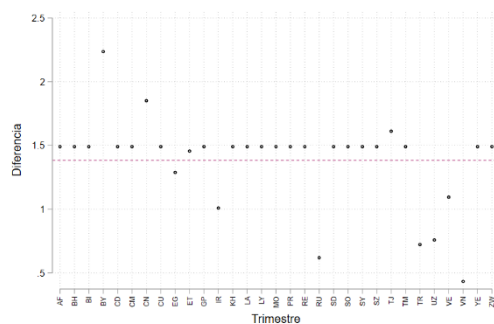
(f) Pesos SDiD estimados

Figura 9

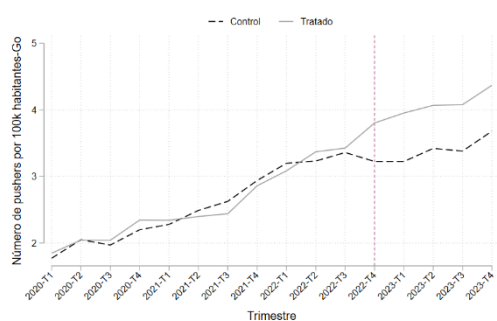
Tendencias y ponderaciones estimadas de Go unique pushers por cada 100 mil personas



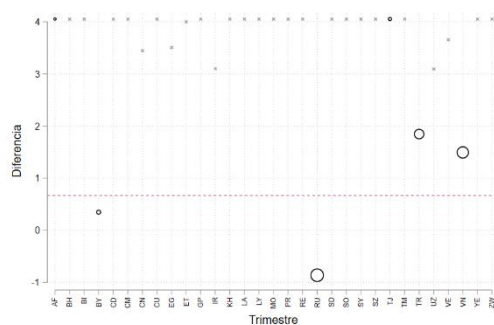
(a) Tendencias estimadas de DiD



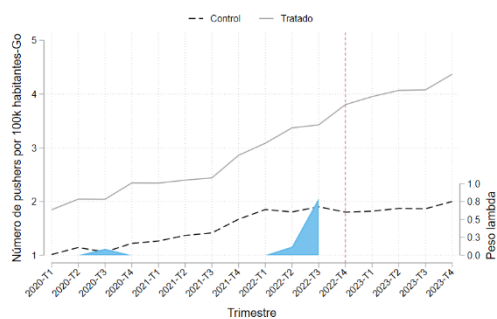
(b) Pesos DiD estimados



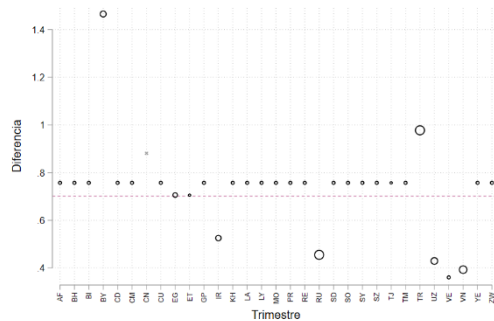
(c) Tendencias estimadas de CS



(d) Pesos CS estimados



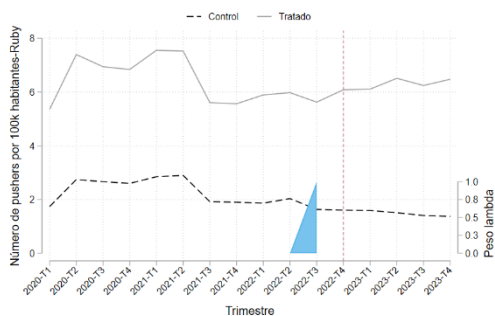
(e) Tendencias estimadas de SDiD



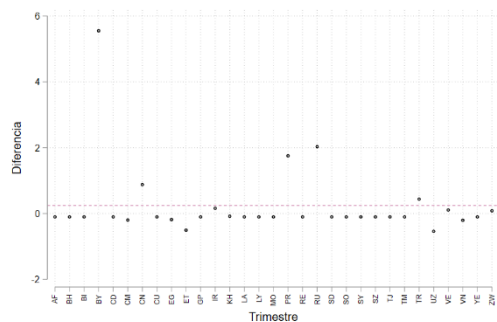
(f) Pesos SDiD estimados

Figura 10

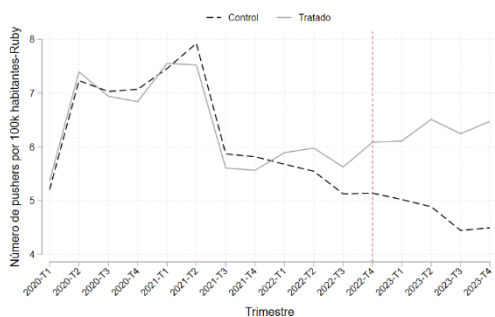
Tendencias y ponderaciones estimadas de Ruby unique pushers por cada 100 mil personas



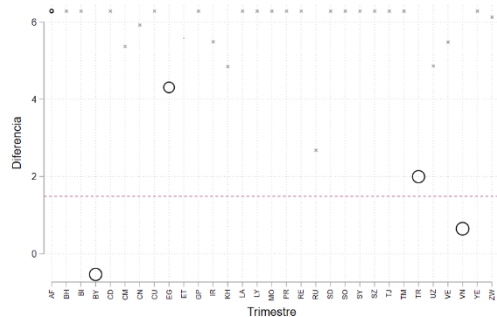
(a) Tendencias estimadas de DiD



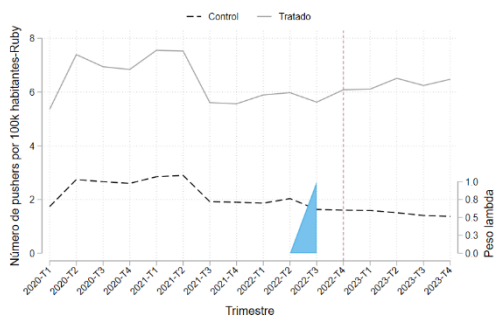
(b) Pesos DiD estimados



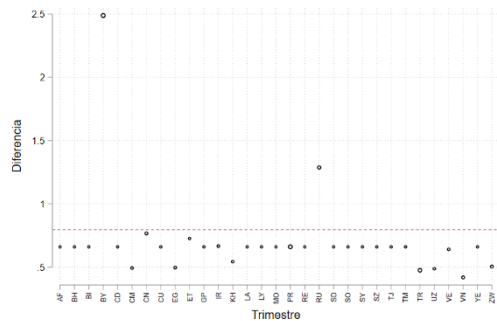
(c) Tendencias estimadas de CS



(d) Pesos CS estimados



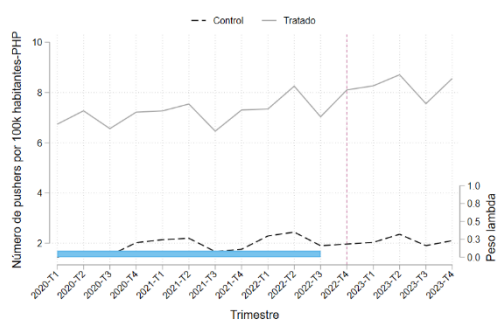
(e) Tendencias estimadas de SDiD



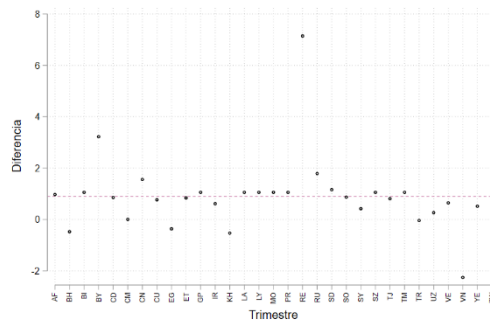
(f) Pesos SDiD estimados

Figura 11

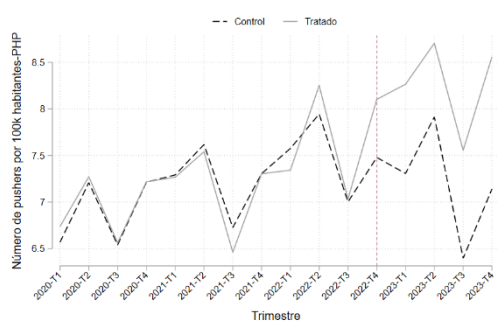
Tendencias y ponderaciones estimadas de PHP unique pushers por cada 100 mil personas



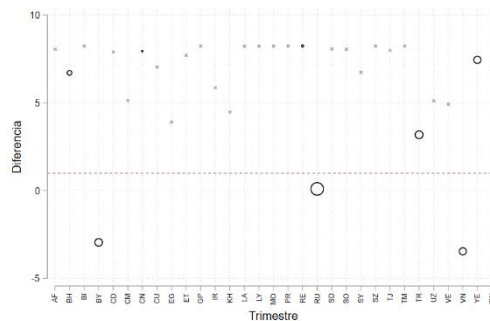
(a) Tendencias estimadas de DiD



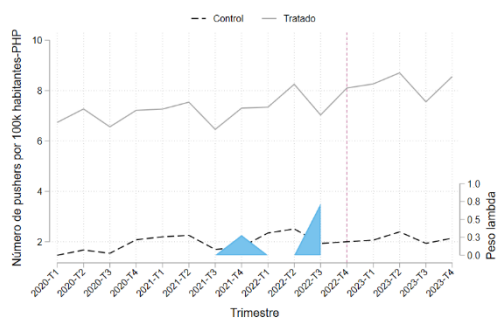
(b) Pesos DiD estimados



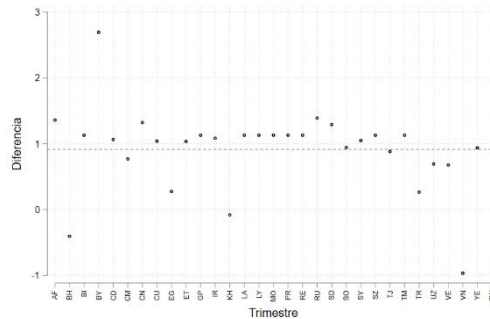
(c) Tendencias estimadas de CS



(d) Pesos CS estimados



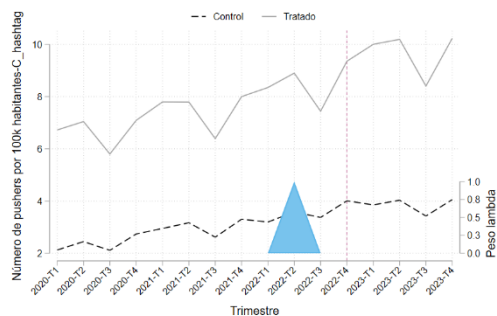
(e) Tendencias estimadas de SDiD



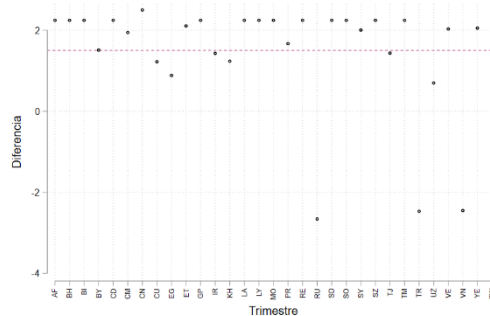
(f) Pesos SDiD estimados

Figura 12

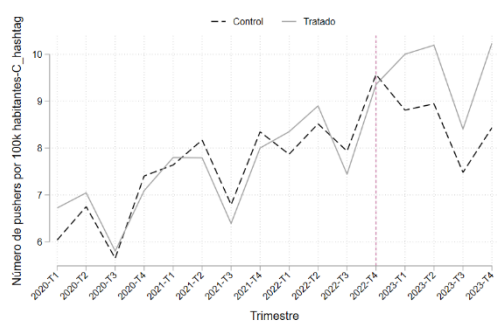
Tendencias y ponderaciones estimadas de C# unique pushers por cada 100 mil personas



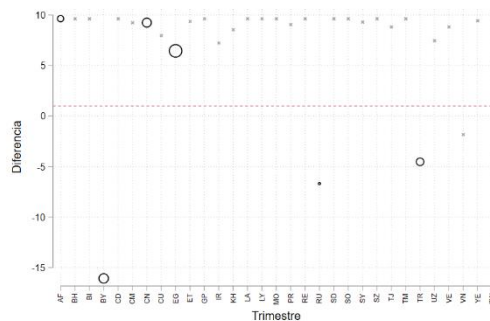
(a) Tendencias estimadas de DiD



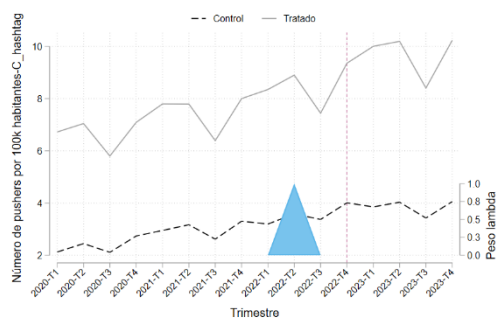
(b) Pesos DiD estimados



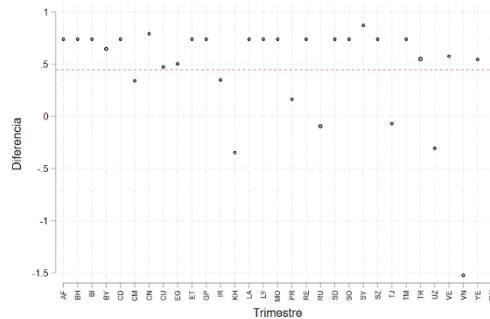
(c) Tendencias estimadas de CS



(d) Pesos CS estimados



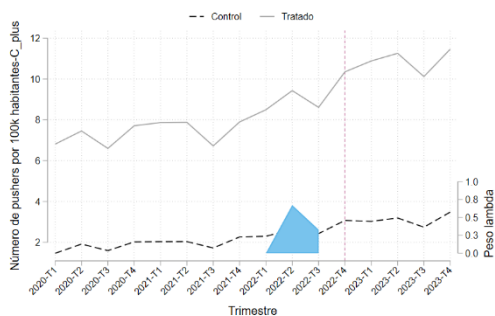
(e) Tendencias estimadas de SDiD



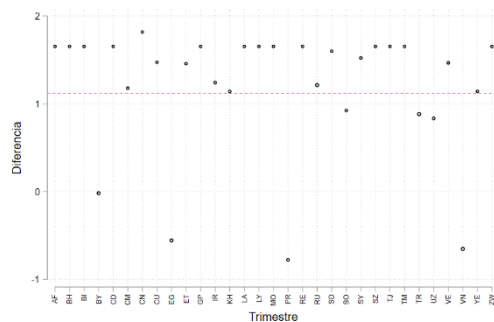
(f) Pesos SDiD estimados

Figura 13

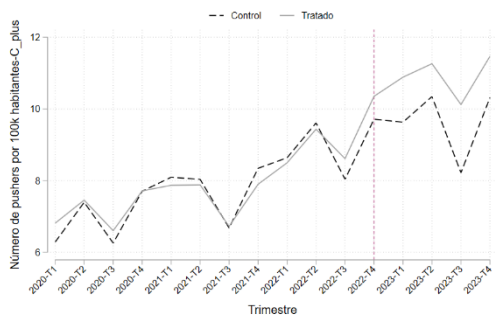
Tendencias y ponderaciones estimadas de C++ unique pushers por cada 100 mil personas



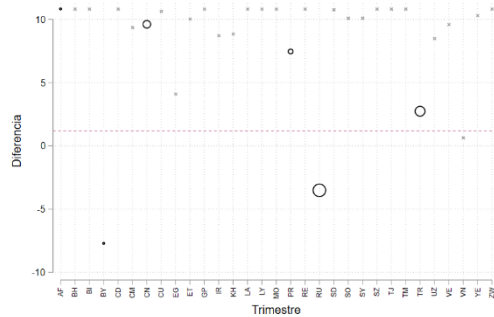
(a) Tendencias estimadas de DiD



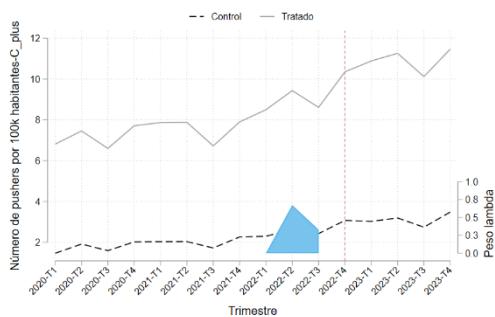
(b) Pesos DiD estimados



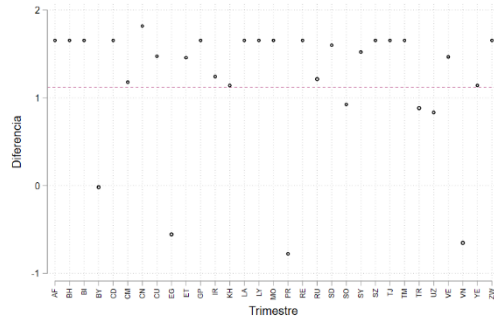
(c) Tendencias estimadas de CS



(d) Pesos CS estimados



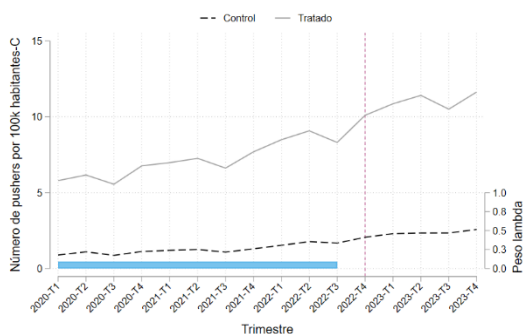
(e) Tendencias estimadas de SDiD



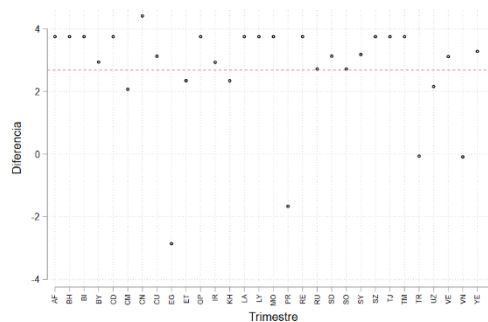
(f) Pesos SDiD estimados

Figura 14

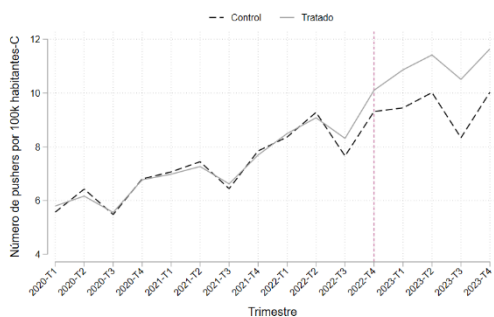
Tendencias y ponderaciones estimadas de C unique pushers por cada 100 mil personas



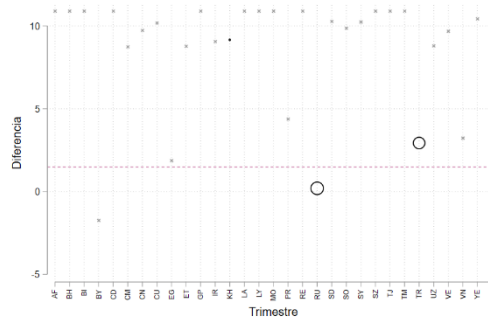
(a) Tendencias estimadas de DiD



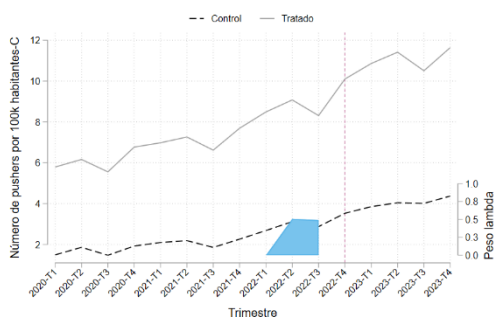
(b) Pesos DiD estimados



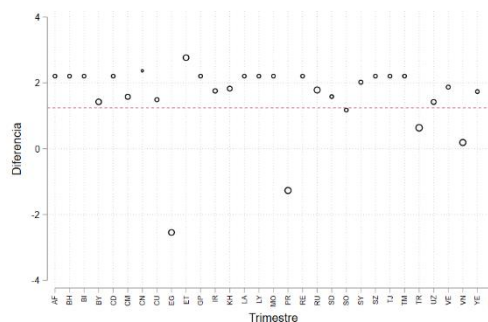
(c) Tendencias estimadas de CS



(d) Pesos CS estimados



(e) Tendencias estimadas de SDiD



(f) Pesos SDiD estimados

10. Anexos

Anexo 1

Listado de países según la disponibilidad de acceso al servicio de ChatGPT

ISO2	País con GPT	ISO2	País sin GPT
AL	Albania	AF	Afghanistan
DZ	Algeria	BH	Bahrain
AD	Andorra	BY	Belarus
AO	Angola	BI	Burundi
AR	Argentina	KH	Cambodia
AM	Armenia	CM	Cameroon
AU	Australia	CN	China
AT	Austria	CD	Congo, The Democratic Republic of the
AZ	Azerbaijan	CU	Cuba
BD	Bangladesh	EG	Egypt
BB	Barbados	SZ	Eswatini
BE	Belgium	ET	Ethiopia
BZ	Belize	GP	Guadeloupe
BJ	Benin	HK	Hong Kong
BT	Bhutan	IR	Iran, Islamic Republic of
BO	Bolivia, Plurinational State of	LA	Lao People's Democratic Republic
BA	Bosnia and Herzegovina	LY	Libya
BW	Botswana	MO	Macao
BR	Brazil	PR	Puerto Rico
BN	Brunei Darussalam	RU	Russian Federation
BG	Bulgaria	RE	Réunion
BF	Burkina Faso	SO	Somalia
CV	Cabo Verde	SD	Sudan
CA	Canada	SY	Syrian Arab Republic
CL	Chile	TJ	Tajikistan
CO	Colombia	TM	Turkmenistan
CG	Congo	TR	Türkiye
CR	Costa Rica	UZ	Uzbekistan
HR	Croatia	VE	Venezuela, Bolivarian Republic of
CY	Cyprus	VN	Viet Nam
CZ	Czechia	YE	Yemen
CI	Côte d'Ivoire	ZW	Zimbabwe
DK	Denmark		
DO	Dominican Republic		
EC	Ecuador		
SV	El Salvador		
EE	Estonia		
FI	Finland		
FR	France		
GA	Gabon		
GM	Gambia		
GE	Georgia		
DE	Germany		
GH	Ghana		
GR	Greece		
GT	Guatemala		
GN	Guinea		
HT	Haiti		
HN	Honduras		
HU	Hungary		

IS	Iceland		
IN	India		
ID	Indonesia		
IQ	Iraq		
IE	Ireland		
IL	Israel		
IT	Italy		
JM	Jamaica		
JP	Japan		
JO	Jordan		
KZ	Kazakhstan		
KE	Kenya		
KR	Korea, Republic of		
KW	Kuwait		
KG	Kyrgyzstan		
LV	Latvia		
LB	Lebanon		
LS	Lesotho		
LR	Liberia		
LT	Lithuania		
LU	Luxembourg		
MG	Madagascar		
MW	Malawi		
MY	Malaysia		
MV	Maldives		
ML	Mali		
MT	Malta		
MR	Mauritania		
MU	Mauritius		
MX	Mexico		
MD	Moldova, Republic of		
MN	Mongolia		
ME	Montenegro		
MA	Morocco		
MZ	Mozambique		
MM	Myanmar		
NP	Nepal		
NL	Netherlands		
NZ	New Zealand		
NI	Nicaragua		
NE	Niger		
NG	Nigeria		
MK	North Macedonia		
NO	Norway		
OM	Oman		
PK	Pakistan		
PS	Palestine, State of		
PA	Panama		
PG	Papua New Guinea		
PY	Paraguay		
PE	Peru		
PH	Philippines		
PL	Poland		

PT	Portugal		
QA	Qatar		
RO	Romania		
RW	Rwanda		
SA	Saudi Arabia		
SN	Senegal		
RS	Serbia		
SC	Seychelles		
SL	Sierra Leone		
SG	Singapore		
SK	Slovakia		
SI	Slovenia		
ZA	South Africa		
ES	Spain		
LK	Sri Lanka		
SE	Sweden		
CH	Switzerland		
TW	Taiwan, Province of China		
TZ	Tanzania, United Republic of		
TH	Thailand		
TG	Togo		
TT	Trinidad and Tobago		
TN	Tunisia		
UG	Uganda		
UA	Ukraine		
AE	United Arab Emirates		
GB	United Kingdom		
US	United States		
UY	Uruguay		
ZM	Zambia		

ANEXO 2

Justificación de la no inclusión de variables de control

En el presente estudio, no se incorporaron variables de control adicionales en las estimaciones realizadas mediante los métodos Difference-in-Differences (DID), Synthetic Control (SC) y Synthetic Difference-in-Differences (SDID). Esta decisión se fundamenta principalmente en la limitada disponibilidad de variables comparables para todos los países incluidos en el análisis. Si bien existen diversas bases de datos internacionales con información potencialmente relevante, como las del Banco Mundial, muchas de ellas no ofrecen una cobertura completa o no son compatibles con la estructura de panel utilizada en esta investigación. La base de datos empleada se encuentra organizada en forma de un panel consistente para un conjunto específico de países y periodos, y al realizar el proceso de imputación para pasar de datos trimestrales a datos anuales, el número de observaciones se redujo de 26 400 a 6 600. Adicionalmente, la incorporación de variables provenientes de fuentes externas implicó nuevas pérdidas de muestras debido a valores faltantes, lo que redujo aún más la muestra disponible. Como se muestra en las dos primeras tablas, inicialmente se produjo la eliminación de tres países (dos pertenecientes al grupo de control y uno al grupo tratado), y posteriormente la presencia de valores vacíos en las variables adicionales generó una reducción aproximada de 340 observaciones adicionales, provocando además un desbalance entre los grupos de tratamiento y control. Esta reducción en el número de unidades disponibles compromete la estabilidad y validez de las estimaciones, especialmente en los métodos SC y SDID, los cuales requieren un conjunto amplio y depurado de países comparables para construir un contrafactual válido.

Mapeo de la cantidad de muestras

Tipo	Cantidad
Quarter	26400
Year	6600
Final	6480

Comparación por grupo de muestras

group	quarter_samples	year_samples	final_samples	lost_q_to_y
0	5120	1280	1200	3840
1	21280	5320	5280	15960

Países eliminados

iso2_code	gpt_available
GF	0
RE	0
TW	1

Missing values o datos vacíos por país

iso2_code	gpt_available_country	n_rows_missing
ET	0	20
HT	1	40
IN	1	20
KE	1	30
PG	1	10
PR	0	20
SD	0	30
SO	0	20
SY	0	40
TM	0	40
VE	0	40
YE	0	30

Impacto de ChatGPT en el número de programadores (con controles)

Lenguaje	DID	SC	SDID	Obs.	Baseline Mean
C	2.453** (0.505)	1.359 (1.281)	1.407** (0.397)	2720	5.829
C#	1.207** (0.341)	0.674 (0.716)	0.471 (0.316)	2720	6.133
C++	2.024** (0.457)	0.818 (0.809)	1.154** (0.411)	2720	6.367
Go	1.192** (0.303)	0.678 (0.504)	0.646** (0.171)	2720	2.086
Java	2.422** (0.674)	1.914* (1.131)	1.690* (0.839)	2720	10.725
JavaScript	11.913** (2.115)	9.199* (5.154)	8.101** (2.505)	2720	34.204
PHP	0.808* (0.336)	0.919 (0.768)	0.923* (0.368)	2720	5.968
Python	6.930** (1.380)	2.122 (3.190)	4.744** (1.158)	2720	18.708
Ruby	0.325 (0.259)	1.482* (0.854)	0.779** (0.265)	2720	5.231
TypeScript	6.094** (1.059)	2.402 (2.392)	2.978** (0.805)	2720	8.017

*Nota. Errores estándar entre paréntesis. * $p < 0.10$, ** $p < 0.05$, *** $p < 0.01$*